

MegaBlocks: Breaking the Logarithmic I/O-Overhead Barrier for Oblivious RAM*

Gilad Asharov[†] Eliran Eiluz[‡] Ilan Komargodski[§] Wei-Kai Lin[¶]

September 4, 2025

Abstract

Oblivious RAM (ORAM) is a central cryptographic primitive that enables secure memory access while hiding access patterns. Among existing ORAM paradigms, hierarchical ORAMs were long considered impractical despite their asymptotic optimality. However, recent advancements (FutORAMa, CCS’23) demonstrate that hierarchical ORAM-based schemes can be made efficient given sufficient client-side memory. In this work, we present a new hierarchical ORAM construction that achieves practical performance without requiring large local memory.

From a theoretical standpoint, we identify that there is a gap in the literature concerning the asymmetric setting, where the logical word size is asymptotically smaller than the physical memory block size. In this scenario, the best-known construction (OptORAMa, J. ACM ’23,) turns every logical query into $O(\log N)$ physical memory accesses (quantity known as “I/O overhead”), whereas the lower bound of Komargodski and Lin (CRYPTO’21) implies that $\Omega(\log N / \log \log N)$ accesses are needed.

We close this gap by constructing an optimal ORAM for the asymmetric setting, achieving an I/O overhead of $O(\log N / \log \log N)$. Our construction features exceptionally small constants (between 1 and 4, depending on the block size) and operates without requiring large local memory. We implement our scheme and compare it to PathORAM (CCS’13) and FutORAMa, demonstrating significant improvement. For 1TB logical memory, our construction obtains $\times 10\text{--}\times 30$ reduction in I/O overhead and bandwidth compared to PathORAM, and $\times 7\text{--}\times 26$ improvement over FutORAMa. This improvement applies when those schemes weren’t designed to operate on large blocks, as in our settings, and the exact improvement depends on the physical block size and the exact local memory available.

Keywords: Oblivious RAM, hierarchical ORAM, tight compaction

*Asharov and Eiluz are supported by the Israel Science Foundation (grant No. 2439/20). Asharov is additionally supported by the European Union (ERC, FTRC, 101043243). Komargodski is supported in part by a grant from the Israel Science Foundation (ISF Grant No. 1774/20), and by the European Union (ERC, SCALE,101162665). Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union or the European Research Council. Neither the European Union nor the granting authority can be held responsible for them.

[†]Gilad.Asharov@biu.ac.il. Department of Computer Science, Bar-Ilan University, Israel.

[‡]eliranfr1@gmail.com. Department of Computer Science, Bar-Ilan University, Israel.

[§]ilank@cs.huji.ac.il. The Hebrew University, Israel.

[¶]wklin@virginia.edu. University of Virginia, USA

Contents

1	Introduction	2
1.1	Our Results	3
1.2	Related Work	6
1.3	Technical Overview	8
2	Preliminaries	10
3	Building Blocks	12
3.1	Oblivious Bin Packing	12
3.1.1	The Functionality	12
3.1.2	The Construction	13
3.2	Non-Recurrent Hash Table	14
3.2.1	The Functionality	14
3.2.2	The Construction	15
3.3	Oblivious Compaction	17
3.3.1	The Functionality	17
3.3.2	Construction	17
4	Oblivious RAM	19
4.1	Overview	19
4.2	The Construction	20
5	Implementation and Evaluation	21
5.1	Evaluation of Our Construction	21
5.2	Comparison to Path ORAM and FutORAMa	23
6	Conclusions	23
A	Additional Preliminaries	32
A.1	Oblivious Simulation of a Non-Reactive Functionality	32
A.2	Oblivious Simulation of a Reactive Functionality	32
B	Supplementary Material for Bin Packing (Section 3.1)	33
B.1	Valid Assignment	33
C	Deferred Proofs	35
C.1	Proof of Theorem 3.6	35
C.1.1	Proof for Theorem 3.9	37
C.2	Proof of Theorem 4.3	38

1 Introduction

An Oblivious RAM (ORAM), introduced by Goldreich and Ostrovsky [Gol87, GO96], is a probabilistic RAM machine that can compile an arbitrary RAM program into a functionally equivalent one, but where the access pattern does not reveal information about the secret inputs or even the program itself. In more detail, an ORAM processes an (online¹) sequence of *logical* RAM operations (**reads** and **writes**) into another sequence of *physical* probes to the memory so that the physical locations being accessed are “independent” of the underlying data. ORAMs have become an indispensable tool in the design of cryptographic systems where it is necessary to perform RAM computation in a privacy-preserving manner. This task comes up not only in the context of software protection, as initially suggested by [GO96], but also in less directly related contexts such as the design of secure processor [FDD12, FRK⁺15], secure multiparty computation [OS97a, LO13, GKK⁺12, GHRW14, WHC⁺14, BCP15], and other central notions in computer science [DMN11, SS13a, SSS12, BNP⁺15, RYF⁺13, MLS⁺13a, GHJR15, LWN⁺15, BCP16, ZWR⁺16, WST12, CKW17].

To see that an ORAM exists, observe the following trivial (but inefficient) construction: upon every logical access, scan the entire memory. This construction suffers from significant overhead: The total number of physical probes to the memory of the ORAM per logical operation is linear in the memory size. The number of physical queries per logical query is a central measure of complexity for ORAM schemes, and it is often referred to as **I/O overhead**. The main question in this context is: how small can this overhead be? We have witnessed significant progress in this front in the past several years, both in constructions and in lower bounds, on which we elaborate next.

Asharov et al. [AKL⁺23b, AKLS23] (improving upon [PPRY18]) obtained a scheme with *logarithmic* I/O overhead in the memory size. That is, they showed a scheme where the overhead in physical accesses per logical query is $O(\log N)$, where N is the logical memory size, consisting of logical words of size $w = \Theta(\log N)$.² This scheme assumes that the size of logical words w and physical blocks b are the same up to constants, i.e., $b \in \Theta(w)$, and that the client has access to trusted memory of size m that can contain a constant number of registers, physical words, i.e., $m = O(1) \cdot b$.

In terms of lower bound, the construction of Asharov et al. [AKL⁺23b, AKLS23] is optimal in their regime of parameters. Indeed, the beautiful lower bound of Larsen and Nielsen [LN18, Theorem 2] says that $\Omega((b/w) \cdot \log(Nw/m))$ I/O overhead is necessary, which simplifies to $\Omega(\log N)$ in the regime considered by Asharov et al.³ Komargodski and Lin [KL21] extended the lower bound of Larsen and Nielsen [LN18] to the asymmetric setting, where b could be significantly larger than w , i.e., $b = \omega(w)$. They showed that any ORAM scheme must have I/O overhead

$$\Omega\left(\log\left(\frac{Nw}{m}\right) / \left(1 + \log\left(\frac{b}{w}\right)\right)\right).$$

¹By “online” we mean that the ORAM supports queries that come in an adaptive manner. There is a notion of *offline* ORAM [BN16] where the compiler gets all accesses at once in advance. Most ORAM schemes are in the online setting, e.g., [GO96, SCSL11, KLO12b, GM11, CGLS17a, SS13b, WCSH16, PPRY18, AKL⁺23a], and there are only a few exceptions [BN16, JLS21, Shi20]. We shall focus exclusively on the online setting in this work.

²Throughout this paper, unless otherwise stated, $\log$ stands for $\log_2$.

³It is worth noting that prior to the lower bound of [LN18], we only knew lower bounds that apply to restricted classes of constructions wherein no cryptography is used and they operated in the “balls and bins” model [BN16]. See Goldreich and Ostrovsky [GO96].

Notice that when $w = \log N$, and $b \in \text{poly } \log N$, i.e., when the server’s block size is polynomially larger than the client’s block size, then the lower bound of [KL21] is $\Omega(\log N / \log \log N)$. No known scheme satisfies sub-logarithmic overhead, leaving a gap between the state-of-the-art constructions and lower bounds.

Asymmetric architectures. Apart from theoretical curiosity regarding the exact complexity of ORAM in the asymmetric setting where $b \gg w$, we argue that this model closely reflects real-world architectures. Memory is structured hierarchically, from processor registers and caches to RAM, hard drives, and cloud storage. As we move down the hierarchy, capacity increases, access times slow, and block sizes grow.

For example, external hard drives and operating systems typically operate at the granularity of pages rather than individual words, and systems do not even support fine-grained word-level access. While CPU registers may operate on 64-bit words, common page sizes are substantially larger—typically, the smallest granularity is 4KB but can be configured to larger blocks (e.g., 64KB, see AArch64 architecture [mmG25]). Consequently, modeling the physical block size as $\text{poly}(\log N)$ instead of $\log N$ offers a reasonable abstraction of systems’ behavior. Moreover, for an adversary, it is often easier to eavesdrop on the physical pages than the CPU words. Our main motivation is to tackle the case where the physical hardware determines the size of the blocks, and those are not necessarily in our control. In such cases, the construction must be adapted to fit the given constraints, rather than freely adjusting parameters to our preference.

Concretely efficient ORAMs. A major challenge is to obtain an ORAM that is *concretely efficient* (see applications in Section 1.2). Currently, the state-of-the-art practical implementations are based on Path ORAM [SS13b]. However, PathORAM requires $O(\log^2 N)$ I/O overhead, which is sub-optimal, leaving hope for improvements. The state-of-the-art asymptotically optimal ORAM, called OptORAMa [AKL⁺23a], achieves $O(\log N)$ I/O overhead but suffers from an enormous leading constant [DO20], rendering it impractical, even for small values of N . A recent work of Asharov et al. [AKM23] gave a concrete, practically useful instantiation of OptORAMa compromising on a rather large local memory, i.e., $m = \Omega(\sqrt{N})$ (and with $w, b \in O(\log N)$). An interesting question is whether relaxing $B = b/w \in \omega(1)$ could serve as an alternative compromise for an asymptotically optimal and practically efficient ORAM.

1.1 Our Results

We close the gap between the lower bound and the upper bound, and, perhaps surprisingly, we show that sub-logarithmic I/O overhead is possible. Indeed, our construction has $O(\log N / \log \log N)$ I/O overhead, assuming that $b/w \geq \log^{1+\Omega(1)} N$, matching the lower bound of [KL21]. In addition, our construction is relatively simple and is highly practical due to exceptionally small leading constants. We provide an implementation of our construction and an elaborate experimental review to support our claims.

Theorem 1.1. *Assume that there is a PRF family that is secure against any probabilistic polynomial-time adversary except with a negligibly small probability in κ . Assume that $\kappa \leq N \leq T \leq \text{poly}(\kappa)$ for any fixed polynomial $\text{poly}(\cdot)$, where T is the number of accesses. Assume, moreover, that $B = b/w \in \Omega(\log^{1+\epsilon} N)$ for constant $\epsilon > 0$.*

Then, there is an ORAM scheme with $O(\log(N/m)/\log B)$ I/O overhead that consumes local memory of m bits and whose security failure probability is upper bounded by a suitable negligible function in κ .

Putting it differently, if the server operates on a suitably larger block size than the client, any program can be simulated obliviously with $o(\log N)$ -overhead. We list additional features of our scheme next.

1. **Optimality:** The construction matches the lower bound of Komargodski and Lin [KL21] for “large blocks,” i.e., when $B = b/w \in \Omega(\log^{1+\epsilon} N)$ and $\epsilon > 0$ is a constant. This results in an overhead of $O(\log(N/m)/\log \log N)$. This complements OptORAMa, which is optimal for the setting where $b = O(w)$ and achieves logarithmic overhead.
2. **Concrete efficiency:** The construction is highly efficient. When relating to the expression $O(\log N/\log \log N)$, the hidden constant is as small as 4, and decreases as B increases: when $B = \omega(\log^4 N)$ the constant drops to just 2. We spent efforts on tightly analyzing the constants and optimizing them. Our experiments demonstrate that the constants are even smaller in practice, in the realm of just 1.

Our analysis accounts for both the read and the subsequent write of the same block (i.e., each client-server round trip involves two accesses). This factor of two appears unavoidable, as all known ORAM schemes write back the block after reading it.

3. **Simplicity:** Our construction follows the hierarchical ORAM framework [?, GO96]. It requires relatively simple machinery compared to OptORAMa [AKL⁺23a], and does not require the sophisticated techniques of maintaining randomly shuffled arrays, interspersing levels, overflow piles, stashes, or cross-level stash aggregation. This makes it easy to implement, debug, and even teach.
4. **External (server) memory size:** There is no overhead in the size of the external memory in the simulated program.
5. **Client’s memory size:** The client’s local memory remains minimal, i.e., it is of size $m = O(1) \cdot b$ bits, containing only a constant number of physical blocks.
6. **Physical block size:** Our construction requires that the physical block would be larger than the logical word size by a factor of $O(\log^{1+\epsilon} N)$. Luckily, this is a lightweight requirement and does not mean extremely large blocks. The minimal physical block sizes that our construction can accommodate is $\approx 4\text{KB}$. In practice, typical logical word sizes are 32–64 bits and the page size is 4KB, which fit our construction.
7. **Amortization:** The overhead of $O(\log N/\log \log N)$ is in an amortized sense. This can be made worst-case using the standard approach of Ostrovsky and Shoup [OS97a] at the expense of roughly 50% extra cost, which comes from more frequent data-structure maintenance, called “Rebuild” later.

Efficiency metrics. Our primary metric is the number of accesses to physical memory, also known as I/O overhead, or “number of probes.” This metric is well-defined and robust across various ORAM settings. Other factors—such as round complexity and computation—are also relevant, interrelated, and have been widely studied. We stress that the lower bounds established in [Gol87, LN18, KL21] specifically focus on this metric.

	Symmetric Setting $B = b/w \in O(1)$		Asymmetric Setting $B = b/w \in \Omega(\log^{1+\epsilon} N)$	
	I/O-overhead	Bandwidth	I/O-overhead	Bandwidth
Path ORAM [SS13b]	$O(\log^2 N)$	$O(b \cdot \log^2 N)$	$O(\log^2(N/B))$	$O(b \cdot \log^2(N/B))$
Circuit ORAM [†] [WCSH16, CS17]	$O(\frac{\log^2 N}{\log \log N})$	$O(b \cdot \frac{\log^2 N}{\log \log N})$	$O(\frac{\log^2 N}{\log B \cdot \log \log N})$	$O(b \cdot \frac{\log^2 N}{\log B \cdot \log \log N})$
OptORAMa [AKL ⁺ 23a]	$O(\log N)$	$O(b \cdot \log(N/B))$	$O(\log(N/B))$	$O(b \cdot \log(N/B))$
Our Work	—	—	$O(\log N / \log B)$	$O(b \cdot \log N / \log B)$

Table 1: A comparison between our construction and previous works in terms of asymptotic efficiency. We emphasize that (1) Our construction is not even defined for the symmetric setting; (2) We consider a natural adaptation of Path ORAM and OptORAMa for the asymmetric setting for which, for utilizing the larger blocks as the server, the construction packs B consecutive logical addresses in one physical address and reduces the address space to N/B . [†]The Circuit ORAM variant is computationally secure and is optimized in the asymmetric setting as per [CGLS18, Theorem 7].

Another common metric is *bandwidth*, which is the number of bits transmitted with each access. Bandwidth and I/O overhead are tightly related, as bandwidth is simply the I/O overhead times the physical block size.

Main technique: dynamic packing. Existing ORAM schemes such as [SS13b, AKL⁺23a, AKM23] are optimized for the symmetric setting, where $w = b$. When $b \gg w$, storing a single logical word in a single physical block is wasteful and results in $O(b \cdot \log N)$ bandwidth. A natural optimization is to *statically* pack $B = b/w$ logical words into each physical block, reducing the address space from N to N/B . Specifically, for every $i \in [N/B]$, the logical addresses $i \cdot B, \dots, (i+1) \cdot B - 1$ are grouped into a single physical block stored at address i . Accessing a logical address addr then corresponds to accessing the block at $\lfloor \text{addr}/B \rfloor$. However, this approach leads to a sub-optimal bandwidth of $O(b \cdot \log(N/B))$.

Our main idea is to utilize *dynamic packing*, where logical words are assigned to physical blocks in a randomized and adaptive manner—e.g., using random binning. This approach yields an improved bandwidth of $O\left(b \cdot \frac{\log N}{\log B}\right)$, which asymptotically outperforms the static packing approach when $B = \omega(1)$. We refer the reader to Section 1.3 for a technical overview of our construction.

We compare the asymptotics of our construction to the previous works in Table 1.

Implementation and experimental evaluation. We implemented the construction and evaluated its performance. Concretely, we provide a full simulation mode of the construction, as well as a “counting mode,” which does not allocate memory but rather computes the number of accesses for large memories. Our experiment validates the theoretical complexity analysis of $4 \cdot \log N / \log \log N$ I/O overhead (and in practice, the constant is even smaller).

We provide a detailed comparison to a reference implementation of Path ORAM [SS13b] and to FutORAMa [AKM23]. In one of our tests, we use a 1TB logical memory, and the results are that our construction reduces the I/O overhead (and bandwidth) significantly. Specifically, we obtain $\times 10$ – $\times 30$ reduction in I/O overhead and bandwidth compared to PathORAM (the exact improvement depends on the physical block size), and $\times 6.5$ – $\times 26$ smaller I/O overhead than FutORAMa. See more details in Section 5, and Tables 6, 7. The experiments and comparisons are conducted on the same parameters (N, w, b) for all schemes, where PathORAM and FutORAMa are instantiated with the aforementioned static packing. We reiterate that our purpose is to improve the efficiency

in large-block settings; in the small-block setting, existing schemes perform better in terms of bandwidth.

Summary of our contribution. We conclude this section with a summary of our contribution:

- Our work focuses on the asymmetric case where the physical block size is asymptotically larger than the logical word size. For that case, we identified a gap in the literature and observed that no optimal construction is known.
- We provide the first asymptotically optimal ORAM construction for those setting, resulting in an I/O overhead of $O(\log N / \log \log N)$. The construction does not consume large local memory and is the first practically efficient hierarchical ORAM that supports large memory sizes with small local memory.
- We provide a thorough theoretical analysis for the overhead cost, and obtain exceptionally small constants in the realm of 2–4 (depending on the block size).
- We implemented our construction. Our experiments confirm the theoretical analysis and demonstrate tiny constants in practice (even smaller than the theoretical analysis, in the realm of just < 1).
- We compared our construction to the reference implementation of Path ORAM, and that of FutORAMa. Our construction outperforms both constructions by significant factors $\times 6.5$ – $\times 26$ for 1TB memory.
- **Open source:** Our code is published as open source, and is available at: <https://github.com/cryptobiu/MegaBlocks>.

1.2 Related Work

Due to its wide applicability, Oblivious RAMs were extensively studied in various setting. A major effort was put into improving the overhead [SS13b, WCSH16, DvDF⁺16, PPRY18, AKL⁺23a], and developing lower bounds [GO96, LN18, KL21].

ORAM and locality. In general, an ORAM cannot achieve locality even if the logical accesses enjoy good spatial locality [ACN⁺17]. Indeed, many applications exhibit good locality. However, every ORAM scheme must hide whether the client requests a sequence of random locations or a contiguous region in the logical accesses. Hence, any scheme that maintains locality inevitably leaks information [ACN⁺17]. Additionally, some applications inherently lack spatial locality: pointer-based data structures (e.g., linked lists, trees, graphs, hash tables) where the next access depends on pointer dereferencing; Basic algorithms like BFS, DFS, binary search, or more complicated algorithms like GNN or SAT-solvers are all pointer-based. While ORAMs cannot enjoy this asymmetry due to obliviousness, we show how to achieve the best in this asymmetry.

Hierarchical vs. tree-based ORAMs. There are two main approaches to constructing oblivious RAM (ORAM). The first, known as hierarchical ORAM, originates from the work of Goldreich and Ostrovsky [Gol87, GO96] and is discussed in Section 1.3. The second, introduced by Stefanov et al. [SCSL11], is the tree-based ORAM approach, where the notable construction in this framework is Path ORAM [SvDS⁺18].

Tree-based ORAMs. Tree-based ORAMs are elegant and practically efficient constructions with small hidden constants [RFK⁺15]. However, they require a poly-logarithmic local memory size and their overhead is $O(\log^2 N)$. Due to its concrete efficiency for moderate memory

sizes, Path ORAM-style systems have been widely implemented and deployed. The Ascend secure processor [FDD12, RFK⁺19], the Phantom secure processor [MLS⁺13a], the GhostRider system [LHM⁺15], and the practical obfuscation project [NFR⁺17], all adopt versions of Path ORAM. More recently, the secure messaging company Signal announced using Path ORAM for a faster layer for enclaves [Sig22]. Despite its practical advantages, as mentioned, Path ORAM requires poly-logarithmic local memory and incurs an $O(\log^2 N)$ I/O overhead (i.e., a total of $O(\log^3 N)$ bandwidth for $O(\log N)$ blocks), which is sub-optimal. Furthermore, it remains an open question whether this framework can achieve an optimal construction that meets known lower bounds [GO96, LN18, KL21].

Path ORAM with large blocks. Path ORAM [SS13b] (and its follow ups, e.g., [WCSH16]) is known to be highly efficient for “large block size,” in which case it achieves $O(\log N)$ bandwidth overhead.

The asymmetry between the block size of the client and the server is in the opposite direction from ours. Specifically, to achieve $O(\log N)$ -bandwidth overhead in Path ORAM, the client’s block size has to be $\Omega(\log^2 N)$, and the external memory has to support “non-uniform block size” – sometimes it needs to access with granularity of $O(\log^2 N)$ block size, and other times with granularity of size $O(\log N)$. This can be simulated using a memory with $O(\log N)$ word size, resulting in a memory with a smaller block size than the client. In terms of I/O, the number of accesses is $\Omega(\log^2 N / \log \log N)$ per client’s block (the term “overhead” could be misleading due to the non-uniform block sizes), as long as the internal memory of the client is $\text{poly}(\log N)$.

Hierarchical ORAM. Hierarchical ORAMs have generally been considered impractical. However, from a theoretical perspective, they represent a significant success. By incorporating several non-trivial ideas and optimizations, the constructions of [PPRY18, AKL⁺23a] achieve an optimal design: for a memory of size N cells, with only $O(1)$ words of local memory and a word size of $O(\log N)$ bits, they attain an $O(\log N)$ overhead—matching known lower bounds.

Despite this theoretical optimality, this approach was widely considered less practical than the tree-based scheme. Originally, this is due to the use of asymptotically-optimal oblivious sorts with high constants [AKS83a] for achieving better ORAM asymptotics, but also continued with recent advances, for instance, the hidden constant in the construction of [AKL⁺20] is huge (around 10,000 [DO20]). A recent construction called FutORAMa [AKM23] provided an optimized version of OptORAMa, assuming that the client’s local memory is relatively large (in the realm of $\sqrt{N}$), performing much of the computing on the client side. Such large local memory might not always be available. Nevertheless, it demonstrates that hierarchical ORAM-based schemes can be made practical under suitable conditions.

Our construction also follows the hierarchical ORAM paradigm, further supporting the claim that a hierarchical ORAM-based scheme can be made practical. Moreover, our construction *does not* rely on a large local memory, and the only requirement is that the client can store $O(1)$ physical words (which we view as $\text{poly}(\log N)$).

Distributed ORAM. A distributed ORAM (DORAM) [LO13, ?] is a multiparty computation protocol for maintaining and accessing a secret shared RAM among mutually distrustful parties. A recent work by [NFO24] presents a 3-party honest majority DORAM scheme with $O(\log N / \log \log N)$ messages assuming poly-logarithmic-size blocks. Their setting is obviously very different from ours: there are multiple servers, most of which are honest, and the servers are allowed to actively perform computation.

Allowing server’s computation is known to be a way to go below the logarithmic barrier for client-server bandwidth in the classical ORAM setting [DvDF⁺16] (this work assumes that the server is evaluating homomorphic encryption). Nevertheless, the I/O overhead (number of server’s memory probes) is $\Omega(\log^2 N)$, even if large blocks are assumed.

1.3 Technical Overview

We follow the hierarchical ORAM framework, originally introduced by Goldreich and Ostrovsky [Gol87, GO96] and later refined in works such as [CGLS17a, GM11, KLO12b, AKL⁺23a]. This framework operates as follows: Given a logical memory of N blocks, we construct a hierarchy of hash tables, denoted $T_0, T_1, T_2, \dots, T_L$, where $L = \lceil \log_2 N \rceil$. Each table T_i , known as the i -th level, stores 2^i logical memory blocks and is implemented as a non-recurrent oblivious hash table. These tables support efficient lookups, provided the same element is not queried twice between rebuilds. Additionally, they allow dummy lookups, ensuring an indistinguishable access pattern regardless of whether a real or dummy element is queried. Each table is also associated with a flag indicating whether it is full or empty.

Upon receiving a request to **read** or **write** a logical memory address **addr**, the ORAM protocol executes the following steps:

1. **Read Phase:** Sequentially access each non-empty level $T_0, \dots, T_L$ and perform **Lookup(addr)**. If the element is found in some level T_i , perform a dummy lookup on all subsequent levels $T_{i+1}, \dots, T_L$.

At the end of the read phase, the value associated with **addr** is successfully retrieved.

2. **Write Back:** If the operation is **read**, store the retrieved data from the read phase in the temporary level **curr** and return it at the end of the operation (after the rebuild phase, coming next). If the operation is **write**, discard the retrieved data and instead **Insert** the new value into **curr**.

3. **Rebuild:** Locate the first empty level T_ℓ for $\ell \geq 0$. If no such level exists, set $\ell = L$, the last level. Merge all levels **curr**, $T_0, \dots, T_{\ell-1}$ into T_ℓ , marking the merged levels as empty and T_ℓ as full. In other words, level T_ℓ is rebuilt using the contents of all preceding levels.

After this step, the temporary level **curr** is always empty. By construction, each level T_i holds 2^i elements when full and alternates between empty and full every 2^i accesses.

Efficiency. If the tables support fast lookups in $O(1)$ time, the read phase completes in $O(\log N)$ as the number of levels. Additionally, level i is rebuilt every 2^i accesses. Previous constructions (e.g., [GO96, KLO12b]) built such hash tables using oblivious sorts, requiring $O(n \log n)$ work to rebuild a level with n items. Summing over all levels resulted in

$$\sum_{i=0}^{\log N} \frac{T}{2^i} O(2^i \log 2^i) = T \cdot \sum_{i=0}^{\log N} O(i) = T \cdot O(\log^2 N) .$$

for T accesses, i.e., $O(\log^2 N)$ overhead per access.

The works of [PPRY18, AKL⁺23a, AKM23, PPY18] improved this by avoiding oblivious sorts when constructing hash tables. Instead, they reused randomness from previous rebuilds and imposed input assumptions, ensuring hash tables remained “randomly shuffled.” Maintaining these

assumptions required techniques such as “interspersing” levels. Additionally, achieving $O(1)$ lookup per level necessitated mechanisms like overflow piles, big and small hash tables, stashes, and cross-level stash aggregation. Ultimately, they reduced the rebuild cost to $O(n)$ per table with n elements, leading to an amortized overhead of

$$\sum_{j=0}^{\log N} \frac{T}{2^j} \cdot O(2^j) = O(T \log N)$$

for T accesses, i.e., an overhead of $O(\log N)$ per access due to rebuilding.

The Challenge. There are two central operations in a hierarchical ORAM that need to be implemented efficiently: (1) lookup and (2) rebuild. The challenge is to optimize both obtain sub-logarithmic I/O overhead, per operation (at least in an amortized sense).

Reducing the number of levels? The first idea for improving the lookup time is to reduce the number of levels to $L = \log N / \log \log N$. If we spend $O(1)$ time per level during lookup time, this would reduce the lookup time to $O(L)$.

However, reducing the number of levels directly means that we need to store the first $\log N - L$ levels in the client’s local memory, requiring storing

$$\sum_{i=0}^{\log N - L} 2^i \geq 2^{\log N - L} = N/2^L \geq N^{1-1/\log \log N}$$

elements, i.e., nearly $O(N)$ local storage. This renders this approach infeasible.

Reducing the number of levels by a larger expansion factor. An alternative approach for reducing the number of levels is to increase the “expansion factor” from 2 to $q \geq \log N$. That is, each level T_i will have capacity of $O(q^i)$ elements, reducing the number of levels from $\log_2 N$ to

$$L = \log_q N = \frac{\log N}{\log q} \leq \frac{\log N}{\log \log N}.$$

Assuming $O(1)$ lookup time per level, this reduces the lookup overhead to $O(\log N / \log \log N)$.

Nevertheless, the rebuild cost increases significantly. With a larger expansion factor, each level must be rebuilt $q - 1$ times before merging into the next level. Moreover, level T_i is rebuilt using the contents of all previous levels $T_0, \dots, T_{i-1}$ *as well as* its own partial content, since it is neither empty nor full between rebuilds. This means that over $T \geq N$ accesses, level i is rebuilt T/q^{i-1} times. Assuming an efficient $O(n)$ rebuild cost per level (which is non-trivial to achieve), the total rebuild overhead is:

$$\sum_{i=0}^{\log N / \log q} \frac{T}{q^{i-1}} \cdot O(q^i) = O\left(\frac{\log N}{\log q} \cdot q \cdot T\right) \geq \left(\frac{\log^2 N}{\log \log N} T\right).$$

Actually, for any $q = \omega(1)$, this results with more than *superlogarithmic* I/O overhead per access. Thus, while reducing the lookup overhead by a $\log \log N$ factor, the rebuild overhead increases by at least $\log N / \log \log N$ multiplicative factor.

Packing within large blocks. To overcome these challenges, we leverage the fact that the logical memory word size w and the physical memory word size b are not equal, with $b \gg w$. Let

$B = \lceil b/w \rceil$, and assume for ease of presentation of this overview that $B = \Omega(\log^2 N)$ and $q = \log N$. Given the server's larger block size, we utilize the idea of **packing**: n logical elements from the client can be stored using within $O(n/B)$ physical blocks. If we can build a level with $O(\frac{n}{B} \log \frac{n}{B})$ physical accesses, then the total rebuild cost is:

$$\sum_{i=0}^L \frac{T}{q^{i-1}} \cdot O\left(\frac{q^i}{B} \log \frac{q^i}{B}\right) \leq \sum_{i=0}^L T \cdot O\left(\frac{q}{B} \log N\right) = T \cdot L = T \frac{\log N}{\log \log N}.$$

The first inequality follows from $\frac{q^i}{B} \leq q^i \leq q^L \leq N$, and the second equality from $q = \log N$ and $B \in \Omega(\log^2 N)$, implying $\frac{q \log N}{B} \in O(1)$. This reduces the rebuild overhead to $O(\log N / \log \log N)$.

Achieving $O(\frac{n}{B} \log \frac{n}{B})$ physical accesses for building hash tables with n logical elements is very challenging since it requires rebuild to make less accesses than the number of logical elements in the hash table. For example, as part of the hash table build in [PPRY18, AKL⁺23a, AKM23], one has to throw the n elements, in the clear (non-obliviously!) to destination bins. In our case, while the n elements are packed into n/B blocks, still writing them into random destination bins requires $\Omega(n)$ accesses, and not the number of physical blocks n/B .

To this end, we leverage **packed oblivious sort**, enabling oblivious sorting (and routing) on packed data in $O(\frac{n}{B} \log \frac{n}{B})$ accesses. In fact, we do not need a full oblivious sort, and use just oblivious bin-packing—a primitive that allows placing elements into destination bins, obliviously. Our bin packing is based on the bucket oblivious sort of [ACN⁺20]. Unlike prior works [PPRY18, AKL⁺23a, AKM23], our approach does not require complex techniques such as stashing, overflow piles, and alike. This not only simplifies the overall design and having small constants, but also makes the construction significantly easier to implement, which we consider an additional advantage. See Section 3.1 for full details.

Organization. The rest of this paper is organized as follows. We give in Section 2 the definitions and the efficiency metrics. In Section 3 we give the underlying building blocks in our construction – oblivious bin packing, oblivious non-recurrent hash tables, and oblivious compaction. In Section 4, we give our ORAM construction. In Section 5 we overview our implementation and give experimental results.

2 Preliminaries

We provide definitions of the RAM model, oblivious computation, and simulation and cover important efficiency metrics. The definitions are based on those in [AKL⁺23a].

Notations. Let X be a distribution. Denote $x \leftarrow X$ as x chosen uniformly at random from X . We denote an array by parentheses, $A = (A_0, \dots, A_{n-1})$, where A_i is an array element. A **distribution ensemble** $X = \{X_n\}_{n \in \mathbb{N}}$ is a sequence of random variables indexed by a security parameter n . We say that a function $\text{negl}(n)$ is **negligible** if for every positive polynomial $p(n)$, there exists an n_0 such that for all $n > n_0$, $\text{negl}(n) < \frac{1}{p(n)}$. For two distribution ensembles X and Y , we consider three types of indistinguishability: *perfect* (X and Y are identically-distributed), *statistical* (the statistical distance between X and Y is negligible, $\text{SD}(X, Y) = \frac{1}{2} \cdot \sum_v |\Pr[X = v] - \Pr[Y = v]| \leq \text{negl}(n)$), and *computational*, i.e., for every Probabilistic Polynomial Time (PPT) distinguisher $\mathcal{D}$ there exists a negligible function $\text{negl}(\cdot)$ such that: $\left| \Pr_{x \leftarrow X}[\mathcal{D}(x) = 1] - \Pr_{y \leftarrow Y}[\mathcal{D}(y) = 1] \right| \leq \text{negl}(n)$.

The RAM model. Random-Access Machine (RAM) is a model of computation, parameterized by $N, w \in \mathbb{N}$, where there are two main components, the CPU and the random access memory. The CPU holds $O(1)$ of registers (each of size w bits). The memory is an array of N blocks, denoted by **mem**, and each block consists of w bits and is indexed by the addresses in $[N] = \{0, \dots, N-1\}$. A RAM program performs the following iteratively. The CPU sends **read** or **write** data from the memory, in the form of $(\text{op}, \text{addr}, \text{data})$, where $\text{op} \in \{\text{read}, \text{write}\}$, $\text{addr} \in [N]$ and $\text{data} \in \{0, 1\}^w \cup \{\perp\}$. The memory returns the result of the command: If $\text{op} = \text{read}$, then $\text{data} = \perp$, and the command returns the content of address **addr** in the memory, **mem**[**addr**]. If $\text{op} = \text{write}$, then **data** is written to **mem**[**addr**]. The CPU continues the next iteration using the returned content until the CPU halts.

Oblivious RAM program. Informally, a RAM program is called *oblivious* if the memory addresses accessed by the program is independent of the input. We often say that the accessed addresses is the *access pattern* of the program for short. For a program M with input x , we denote by $\text{AccessPtrn}(M(x))$ all the memory locations that the program accesses in its execution. Note that this is a distribution when M is randomized.

In Appendix A, we review the definition of reactive and non-reactive functionalities. We proceed with the definition of an oblivious RAM compiler.

Oblivious RAM compiler. An oblivious RAM Compiler C is a compiler that takes as an input a RAM machine M and outputs an oblivious RAM machine M' , where M' obviously simulates M according to Definition A.1.

Definition 2.1 (Oblivious RAM Compiler). *Let M be a RAM machine and C an efficient RAM machine such that $C(M) = M'$. C is called an Oblivious RAM Compiler if there exists a PPT simulator $\mathcal{S}$ such that for every input x :*

$$\{M'(x), \text{AccessPtrn}(M'(x))\} \equiv \{M(x), \mathcal{S}(|x|)\}$$

Again, $\equiv$ might refer to statistical and computational indistinguishability.

Efficiency metrics. ORAM is a compiler that receives a RAM program M as an input and outputs another program M' . In the most general setting, the two RAM programs might operate on different word-size, and different memory sizes. Recall that $M[N, w]$ denotes a RAM model where N is the number of memory cells in the memory, w is the word size. We consider a compiler that takes as input a program in $M[N, w]$ and outputs another program in $M'[N', b]$. We often say that the original program M works on the *logical* memory $M[N, w]$, where as the compiled program M' works on the *physical* memory $M'[N', b]$. The logical and physical memory are sometimes referred to as the client and server, correspondingly. Both programs should contain $O(1)$ words (either w or b , respectively). However, we sometimes allow the compiled program to utilize larger internal memory, which we denote its size by m (in bits). We consider the following efficiency metrics:

- **I/O overhead:** Recall that $\text{AccessPtrn}(M(x))$ is the sequence of all Read/Write operations to the memory in the execution of M on input x . The I/O overhead is defined as the maximum number of accesses of the machine M' compared to M over all possible inputs:

$$\text{I/O-overhead} \stackrel{\text{def}}{=} \max_x \frac{|\text{AccessPtrn}(M'(x))|}{|\text{AccessPtrn}(M(x))|}.$$

Note that we count here the ratio in *the number of accesses* to the memory.

- **Bandwidth/Communication overhead:** Defined as the maximum number of bits transferred by M' compared to M . This is exactly $\text{l/O-overhead} \cdot b/w$, also when $b = w$, this equals to l/O-overhead .

Notations. We summarize the notations in Section 2. Looking ahead, we shall build a hierarchical ORAM, and our different building blocks operate on a various number of elements, denoted n (as opposed to N which is the total number of elements). n/B denotes the number of server blocks needed to store these n elements. Whenever B does not divide n , we abuse notation and just use n/B and really mean $\lceil n/B \rceil$. We summarize the notations we use below:

Parameter	Description
N	total number of cells in the logical memory
w	word size of a cell in the logical memory (in bits)
N'	total number of cells in physical memory
b	block size of a cell in the physical memory (in bits)
m	size of the local memory of the compiled oblivious RAM machine (in bits)
B	$B = b/w$: number of logical words that can be packed in a single physical block
n	number of logical elements in a building block
n/B	$n/B = \lceil n/B \rceil$ – number of server blocks needed to store n logical's elements

3 Building Blocks

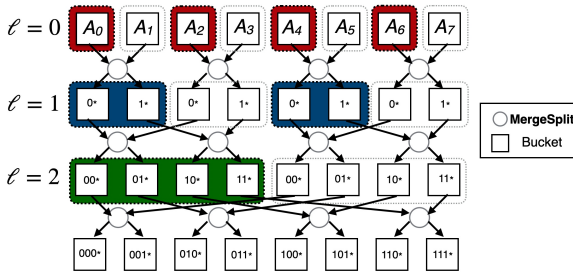
In this section, we overview the necessary building blocks needed for our construction:

- **Oblivious bin packing (Section 3.1):** This oblivious algorithm receives as input n elements, where each is assigned to a destination bin. It outputs bins along with their assigned elements, *obliviously*.
- **Non-recurrent hash table (Section 3.2):** This oblivious data structure implements the logic of a “level” in our construction. It is initialized with n and supports fast lookups. Obliviousness is preserved as long as all lookup queries are for distinct elements. Dummy lookups are also supported.
- **Oblivious compaction (Section 3.3):** This oblivious algorithm receives as an input an array with n elements where $n_0 < n$ of them are marked as “real.” It outputs all real elements.

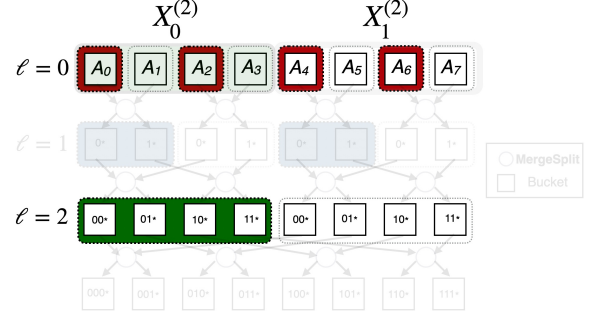
3.1 Oblivious Bin Packing

3.1.1 The Functionality

Bin packing gets as input an array of elements, each is assigned with a destination bin, and the procedure places the elements into their destination bins – obliviously. The algorithm successfully places the elements only if the assignments are valid, as we formalize in Definition B.1. We note that both the functionality and the construction are deterministic.



(a) Bin packing into $C = 8$ buckets. In the ℓ th iteration, the elements are semi-sorted according to the ℓ MSB. MergeSplit then routes its input elements (all share the same ℓ MSB) according to their $\ell + 1$ MSB of their bin assignment.



(b) This demonstrates the “super-bin” for iteration $\ell = 2$, and the corresponding requirement from the $X_0^{(2)}, X_1^{(2)}$, it is required that there are at most B elements with MSB 00, 01, 10, 11, respectively.

Figure 1: Bin Packing – demonstration of the algorithm for $C = 8$, and on the valid assignment.

Functionality 3.1: $\mathcal{F}_{\text{BinPack}}$ – Bin Packing Functionality

- **Input:** An array $X = (a_0, \dots, a_{2n-1})$, packed in C server memory blocks, where each element $a_i = (\alpha_i, d_i)$, where $\alpha_i \in [C]$ is the assigned destination bin, and $d_i \in \{0, 1\}^{O(1) \cdot w}$ is the data. At least n elements are dummies and have no destination bin, i.e., $\alpha_i = \perp$.
 - **Assumption:** It is assumed that the assignment is valid, i.e., the predicate $P((i, \alpha_i)_{i=0}^{2n-1}) = 1$, where the predicate P is defined in Definition B.1.
 - **Output:** An array $Y = (Y_0, \dots, Y_{C-1})$ containing C memory blocks, where each element $a_i = (\alpha_i, d_i) \in X$ resides in Y_{α_i} . It is assumed for simplicity that C is a power of 2.
-

3.1.2 The Construction

The construction is based on “bucket oblivious sort” of [ACN⁺20]. The main difference is that in our setting, since the blocks on the server are “large,” each bin can reside in one (or $O(1)$) server block. In a sense, the algorithm routes the elements through a butterfly network to their assigned bins according to their assignment. The client reads elements from two buckets that are distance 2^i away in level i , and writes them to two adjacent bins in level $i + 1$, using the $(i + 1)$ th bit of each element’s assignment to make the routing decision. See Fig. 1.

Construction 3.2: Bin Packing

- **Input:** The input is an array $X = (a_0, \dots, a_{2n-1})$ packed in C server memory blocks, where each element $a_i = (\alpha_i, d_i)$ with $\alpha_i \in [C]$ is the assigned destination bin, and $d_i \in \{0, 1\}^{O(1) \cdot w}$ is the data. At least n elements are dummies and have no assigned destination bin, i.e., $\alpha_i = \perp$. For simplicity of exposition, it is assumed that C is a power of 2.
- **The algorithm:**

1. Allocate $m = \lfloor \log C \rfloor + 1$ arrays of size C each, denoted as $A^{(0)}, \dots, A^{(m-1)}$, where for every $\ell \in \{0, \dots, m-1\}$ we let $A^{(\ell)} = (A_0^{(\ell)}, \dots, A_{C-1}^{(\ell)})$. That is, $A_j^{(\ell)}$ denotes the j th (server) memory block of the ℓ th array, which is of size b .
 2. Let $A_j^{(0)} = (a_{j \cdot B}, \dots, a_{(j+1) \cdot B-1})$ for every $j \in \{0, \dots, C-1\}$. This is just renaming, and there is no need for actual placement.
 3. For $\ell \in \{0, \dots, m-1\}$ and for every $j \in \{0, \dots, C/2-1\}$:
 - (a) Read bins $A_{j'+j}^{(\ell)}, A_{j'+j+2^\ell}^{(\ell)}$ to (Y_0, Y_1) in the local memory, where $j' = \lfloor j/2^\ell \rfloor \cdot 2^\ell$.
 - (b) Perform locally $(Z_0, Z_1) \leftarrow \text{MergeSplit}_\ell(Y_0, Y_1)$, which is described by the following steps:
 - i. Z_0 gets the real elements $(\alpha, d) \in Y_0 \cup Y_1$ such that the $(\ell+1)$ -th MSB of the key α is 0.
 - ii. Z_1 gets the real elements $(\alpha, d) \in Y_0 \cup Y_1$ such that the $(\ell+1)$ -th MSB of the key α is 1.
 - iii. If either Z_0 or Z_1 exceed the capacity B of a memory block, then **abort**. This occurs only if the assignment is invalid. Pad (Z_0, Z_1) to size B with dummies.
 - (c) Write Z_0 to $A_{2j}^{(\ell+1)}$ and Z_1 to $A_{2j+1}^{(\ell+1)}$.
 - **Output:** The arrays $(Y_0, \dots, Y_{C-1}) = (A_0^m, \dots, A_{C-1}^m)$.
-

We refer the reader to Appendix B, where we formally define valid assignments and prove the following Theorem:

Theorem 3.3 (Oblivious bin packing). *Construction 3.2 perfectly obviously implements Functionality 3.1 for every input X with valid assignment (see Definition B.1), and it accesses $2C \log C$ number of blocks, where $C = 2n/B$.*

3.2 Non-Recurrent Hash Table

3.2.1 The Functionality

The hash table functionality supports three operations:

- **Build(X):** This operation takes as input an array X of key-value pairs (k, v) and organizes them in memory. We emphasize that X resides in the server memory, where n client elements can be stored in n/B server blocks, and we assume that X is already in “packed” form.
- **Lookup(k):** This operation retrieves the value v associated with the key k , if it exists. Additionally, it supports dummy lookups.
- **Extract():** This operation returns the entire array X , with the added property that all previously queried elements are replaced with dummy entries.

The formal definition of the primitive follows.

Functionality 3.4: $\mathcal{F}_{\text{HT}}$ for n elements

$\mathcal{F}_{\text{HT}}.\text{Build}(X)$

- **Input:** X consists of n/B memory blocks, each containing B client’s elements. That is, $X = (X_0, \dots, X_{n/B-1})$, where each X_i is a memory block, consisting of B items. We let $(a_0, \dots, a_{n-1})$

denote all the items. Each item a_i is a pair of the form $(k_i, v_i) \in \{0, 1\}^w \times \{0, 1\}^w$, some of which might be $\perp$ (dummy).

- **Output:** Build stores X and has no output.

$\mathcal{F}_{\text{HT}}.\text{Lookup}(k)$:

- **Input:** $k \in \{0, 1\}^w \cup \{\perp\}$, where $\perp$ denotes a dummy lookup.
- **Output:** If there exists k such that $(k, v) \in X$, then return v and replace (k, v) from X with $\perp$ (dummy). Otherwise, return $\perp$.

$\mathcal{F}_{\text{HT}}.\text{Extract}()$:

- **Input:** The command has no input.
 - **Output:** The command returns X , with the same ordering of the elements as in Build (while some elements were switched to $\perp$ during Lookup).
-

3.2.2 The Construction

When the Build operation is invoked, the construction assigns to each element a random bin, using a PRF key sk . That is, each element (k_i, v_i) is mapped to the bin $\alpha = \text{PRF}_{\text{sk}}(k_i)$. Then, the algorithm performs “bin-packing” (Section 3.1) – it routes the elements into the bins, while also introducing some dummy elements which guarantees that the bin assignments are valid with overwhelming probability (see Definition B.1). Note that each memory block essentially serves as a bin, if we choose the parameters carefully. We also store next to each item its original location in the array X , which will be useful for implementing the Extract() procedure. The access pattern of this procedure is just an invocation of bin-packing, and therefore is oblivious.

In Lookup(k), we visit the bin $\alpha = \text{PRF}_{\text{sk}}(k_i)$, and search for the key k in it. If k is a dummy, then the construction visits a random bin. According to the functionality of $\mathcal{F}_{\text{HT}}$, it is guaranteed that there are no recurrent queries – namely, the same key is not searched twice – and therefore, the access pattern of the lookup phase is distributed according to a “balls into bins” distribution.

Finally, the Extract procedure reverse-routes all elements from their assigned bins back to the array X , while also getting rid of the dummy elements that were introduced. This is possible since we store the original location of each element, and since we can revert the operations of the bin-packing algorithm.

Construction 3.5: Oblivious Non-Recurrent Hash Table

HT.Build(X):

- **Input:** An array X of a total of n elements consisting of n/B memory blocks, each containing B client’s elements. That is, $X = [X_0, \dots, X_{n/B-1}]$ where each X_i is a memory block, consisting of B items. We let $(a_0, \dots, a_{n-1})$ denote all the items. Each item a_i is a pair of the form $(k_i, v_i) \in \{0, 1\}^w \times \{0, 1\}^w$, some of which might be dummy ($\perp$).
- **The algorithm:**
 1. Define $C = 2n/B$. Recall that $n/B = \lceil n/B \rceil$ and that if C is not a power of 2, we round it to the next power of 2.
 2. Sample a PRF key $\text{sk} \in \{0, 1\}^\kappa$.

3. For every $i = 0, \dots, n-1$ assign to $a_i = (k_i, v_i)$ the bin $\alpha_i = \text{PRF}_{\text{sk}}(k_i) \bmod C$. If $k_i = \perp$, assign to α_i a random value in $[0, \dots, C-1]$.
4. Evenly divide the elements in X into $2n/B$ groups of size $B/2$ each, pad each group with $B/2$ dummy elements, and additionally append $C - 2n/B$ dummy groups (of size B each). Denote the result $X' = (X'_0, \dots, X'_{C-1})$. Each real element a_i is written into its group X'_j as $(k_i, v_i, \alpha_i, j, r)$, where $j = \lfloor i/(2n/B) \rfloor$, i.e., the index of the group where the element currently resides in, and $r := i \bmod (B/2)$ (i.e., the element's block index in its block).
5. Call the bin-packing algorithm (Construction 3.2) on X' while treating α_i as the assigned destination bins. Receive back $(\text{Bin}_0, \dots, \text{Bin}_{C-1})$ and store the bins with sk as the internal state.

HT.Lookup(k):

- **Input:** An address $k \in \{0, 1\}^w \times \{\perp\}$.
- **The algorithm:**
 1. Let $(\text{Bin}_0, \dots, \text{Bin}_{C-1}, \text{sk})$ be the secret state defined by Build(X).
 2. If $k = \perp$, sample a random $\alpha \leftarrow [0, \dots, C-1]$. Reach Bin_α and write it back. Set $v^* = \perp$.
 3. Otherwise, let $\alpha = \text{PRF}_{\text{sk}}(k) \bmod C$. Read bin Bin_α , look for $(k, v, \alpha, \cdot, \cdot)$ in this bin. Mark the value as **accessed**, write the bin Bin_α back, and set $v^* = v$. If not found, write the bin back and set $v^* = \perp$.
- **Output:** The value v^* .

HT.Extract().

- **Input:** The command has no input.
- **The algorithm:**
 1. For each $\text{Bin} \in \text{Bin}_0, \dots, \text{Bin}_{C-1}$ that is stored in the secret state, write on each element $a_i = (k_i, v_i, \alpha_i, j, r)$ its destination bin as j . Note that elements that were accessed during lookups are marked as **accessed**, but are treated as real elements.
 2. Reverse route Construction 3.2, namely, move all elements from their destination bins back to the original bins by reversing all choices made in Construction 3.2. Note that This is possible due to storing next to each element its original group it came from. Let $Y_0, \dots, Y_{C-1}$ be the output of the functionality.
 3. Truncate the last $C - 2n/B$ dummy groups $Y_{C-2n/B}, \dots, Y_{C-1}$.
 4. Allocate an array X' of size n/B blocks.
 5. For every $i \in \{0, \dots, n/B - 1\}$:
 - (a) Read Y_{2i}, Y_{2i+1} to the local memory. Arrange the internal ordering in the bin according to the original ordering in X' (using r). Replace elements that are marked as **accessed** with dummies.
 - (b) The last- $B/2$ elements in each bin Y_{2i}, Y_{2i+1} are dummies. Remove those dummies and concatenate Y_{2i}, Y_{2i+1} into a single bin X'_i .
- **Output:** The array $X' = (X'_0, \dots, X'_{n/B-1})$.

Theorem 3.6. Assuming security of PRF, construction HT (Construction 3.5) implements $\mathcal{F}_{\text{HT}}$ (Functionality 3.4). For an input X of size n elements, each of size w bits:

- Build(X) requires accessing $2C \log C + 1.5C$ memory blocks.
- Lookup(k) requires accessing 2 memory blocks (read and write of the same block).
- Extract() requires accessing $2C \log C + 1.5C$ memory blocks.

where C is the closest power of 2 that upper bounds $2n/B$. When modeling the PRF as a random function, the error probability of the construction is $2C \log C \cdot e^{-B/6}$.

We prove this theorem in Appendix C.1.

3.3 Oblivious Compaction

In compaction, the input is an array that consists of n elements, each of which is either “real” or “dummy”, and there are at most $n_0 < n$ real elements. The desired output is an array of n_0 elements that includes all the real elements. Oblivious compaction serves as a fundamental building block in ORAM schemes, such as [AKS83a, Goo11, CGLS18, AKL⁺23a]. In the literature, there are some variants of compaction. Namely, a compaction is called *tight* if all real elements are in the front of the output array [Goo11]. The above notion of compaction is weaker than tight compaction since it is tight only when there are exactly n_0 real elements. In other words, in our case, the quantity n_0 is known to the adversary, and the primitives guarantees that the all the real elements reside in the first n_0 locations of the output. We simply call our variant **compaction**.

3.3.1 The Functionality

The functionality is defined as follows.

Functionality 3.7: $\mathcal{F}_{\text{Compaction}}$

- **Input:** An array X consists of $\frac{n}{B}$ memory blocks, each containing B client’s items. That is, $X = (X_0, \dots, X_{\frac{n}{B}-1})$, where each X_i is a memory block, consisting of B items. We let $(a_0, \dots, a_{n-1})$ denote all the items, where $a_i \in \{0, 1\}^w \cup \{\perp\}$ for all i , where $a_i = \perp$ denotes a *dummy* item. There are at most $n_0 < n$ items are *real*, i.e., in $\{0, 1\}^w$.
 - **Output:** The array Y of length n_0/B memory blocks such that Y consists of all real items in X (and potentially some dummy items). Moreover, all items in Y are shuffled uniformly at random.
-

We emphasize that the above functionality *shuffles* the output array randomly. The purpose of this is to make our construction more efficient and the proof simpler. Later, our final ORAM works with any ordering of the real elements in the output, and therefore the ordering will not be important.

3.3.2 Construction

The compaction algorithm uses Oblivious Bin Packing (Theorem 3.3) as follows. We randomly assign the n_0 elements in X into $C = 2n/B$ bins, and the elements are routed into bins using bin packing. Then, we simply move the assigned elements from each bin to the output array of n_0 elements in the clear. Since the loads of the bins follow a distribution of throwing n_0 balls into C bins uniformly at random, it is safe to reveal the bin loads.

Construction 3.8: Compaction

- **Input:** An array X consists of $\frac{n}{B}$ memory blocks, each containing B client's elements. That is, $X = (X_0, \dots, X_{\frac{n}{B}-1})$, where each X_i is a memory block, consisting of B items. Let $(a_0, \dots, a_{n-1})$ denote all the items, and each item is $a_i \in \{0, 1\}^w \cup \{\perp\}$, some of which might be dummy (in which case, $a_i = \perp$). There is an additional public parameter $n_0 \leq n$ such that there are at most n_0 number of real items in X .
 - **The Algorithm:**
 1. Define $C = 2n/B$. If C is not a power of 2, we round it to the next power of 2.
 2. For each real item, assign a random bin in $[C]$. Let n_r be the number of real items. Among the dummy items, assign $n_0 - n_r$ of them to random bins in $[C]$, and assign the remaining to $\perp$. That is, for each $i = 1, \dots, n-1$,
 - If a_i is real or in the first $(n_0 - n_r)$ dummy items, sample $\ell_i \leftarrow [C]$ uniformly at random; otherwise, let $\ell_i := \perp$.
 3. Evenly divide the pairs (ℓ_i, a_i) in X into $2n/B$ groups of size $B/2$ each, pad each group with $B/2$ dummy pairs, $(\perp, \perp)$. Additionally, pad $C - 2n/B$ groups of dummy pairs, each group is of size B . Denote the result $X' = (X'_0, \dots, X'_{C-1})$.
 4. Call the functionality $\mathcal{F}_{\text{BinPack}}(X')$ (Functionality 3.1) and let $Y_1, \dots, Y_C$ be the resulting array.
 5. Set $\text{currBin} = 0$, $\text{currIndex} = 0$. Allocate an array $A = (A_0, \dots, A_{n_0/B-1})$ of n_0/B groups. Set localBin in the local memory as one block of size B .
 6. For each $i \in \{0, \dots, C-1\}$, shuffle the elements in Y_i uniformly at random. That is, Y_i is read into the local memory, shuffled locally, and then written back.
 7. For each non-dummy pair (ℓ, a) , move the item a to the next slot of A . That is, for each $i \in \{0, \dots, C-1\}$ and for every pair (ℓ, a) in Y_i ,
 - (a) If $\ell \neq \perp$, write a to $\text{localBin}[\text{currIndex}]$ and increment currIndex .
 - (b) If $\text{currIndex} = B$, then write localBin to A_{currBin} , increase currBin by 1, set $\text{currIndex} = 0$, and then set localBin to empty structure.
 8. Pad the remaining of A with dummy items as follows. If $\text{currIndex} < B$, pad localBin with dummies to size B , and write it to A_{currBin} , and then increase currBin by 1. Also, pad the array A with $n_0/B - \text{currBin}$ number of dummy blocks to have size n_0/B .
 - **Output:** The array $A = (A_0, \dots, A_{n_0/B-1})$.
-

The compaction algorithm consists of two phases. In the first phase (Steps 1-4), we assign n_0 elements in X into $C = 2n/B$ bins uniformly at random, including all real elements and potentially some dummy. The access pattern is determined only by the input size n , and the resulting load of bins is independent of the input. In the second phase (Steps 5-8), we move the n_0 elements “in the clear” to the output array. That reveals the loads of bins but is oblivious since the loads depend only on n_0 . Consequently, the output array is effectively compacted. The following theorem is proven in Appendix C.1.1.

Theorem 3.9. *Compaction (Construction 3.8) obviously implements functionality $\mathcal{F}_{\text{Compaction}}$ (Functionality 3.7) with error probability that is bounded by $2C \cdot \log C \cdot e^{-\frac{B}{6}}$, and it accesses $2C \log C + 3C + \frac{n_0}{B}$ number of memory blocks, where $C = \frac{2n}{B}$.*

4 Oblivious RAM

In this section, we provide the full description of our oblivious RAM. We start with presenting Functionality 4.1, proceed with an overview in Section 4.1, and the construction in Section 4.2, deferring its analysis to Appendix C.2.

The functionality. The functionality of ORAM is reactive, and is essentially storing and retrieving data from an array.

Functionality 4.1: $\mathcal{F}_{\text{ORAM}}$

The functionality is reactive, and holds an internal state – N memory blocks, each of size w . Denote the internal state $\mathbf{X}[1, \dots, N]$. Initially, $\mathbf{X}[\text{addr}] = 0$ for every $\text{addr} \in [N]$

- **Access**($\text{op}, \text{addr}, \text{data}$): where $\text{op} \in \{\text{read}, \text{write}\}$, $\text{addr} \in [N]$, and $\text{data} \in \{0, 1\}^w$.
 1. If $\text{op} = \text{read}$, set $\text{data}^* = \mathbf{X}[\text{addr}]$.
 2. If $\text{op} = \text{write}$, set $\mathbf{X}[\text{addr}] := \text{data}$ and $\text{data}^* = \text{data}$.
 3. Output data^* .
-

4.1 Overview

We provide an overview of our construction; the reader is also advised to recall the explanation given in Section 1.3.

We consider the general case where the word size of the client memory w and the block size of the server memory b satisfy $b \gg w$ and denote $B = \lceil b/w \rceil$. Here are the main modifications we apply to the classical Hierarchical ORAM data structure:

- **Structure:** Instead of storing the elements in levels with doubling size, we increase the expansion parameter to $q > 2$ (to be chosen later). At each level T_i , we store $(q - 1) \cdot q^i$ logical words, meaning that each level consumes $O((q - 1) \cdot q^i / B)$ physical blocks. The total number of levels is $L = \lfloor \log_q N \rfloor$. The capacity of T_L is set to N elements and is stored in $O(N/B)$ physical memory blocks. For readability, we assume that N is a perfect power of q and $N = q^L$. We will remove this assumption in Remark C.4.
- **Access:** Access is performed in a similar manner as before – we access each level, searching for the element addr . Once found, we write it back to the temporary level curr (which stores 1 logical word and supports **Insert** similarly).
- **Rebuild:** Recall that in standard hierarchical ORAM, each level T_i stores 2^i elements. Also, each level T_i is rebuilt every 2^i accesses, and the rebuild switches T_i between two states—either *empty*, or *full*. Notice that the first L flags and thus the corresponding levels represent a base-2 integer in $\{0, \dots, 2^L - 1\}$.

In our construction, the loads of all L levels $(T_0, T_1, \dots, T_{L-1})$ represent a base- q integer analogously. Each level T_i (of total capacity $(q - 1) \cdot q^i$) has q states so that T_i stores $0, q^i, 2q^i, \dots$, or $(q - 1) \cdot q^i$ number of elements. In particular, the level T_i is rebuilt every q^i accesses, from the content of all levels $\text{curr}, T_0, T_1, \dots, T_{i-1}$, and also T_i . Hence, we associate with each level the variable $\text{Load}_i \in \{0, 1, \dots, q - 1\}$, where $\text{Load}_i = 0$ means that the level T_i is empty (and is not built), and $\text{Load}_i \in \{1, \dots, q - 1\}$ means that the table is built and containing $\text{Load}_i \cdot q^i$ elements. By construction, when $\text{Load}_i = q - 1$, in the next q^i accesses, we will rebuild the next level, T_{i+1} .

When the first L levels are full, i.e., $\text{Load}_j = q - 1$ for all $j < L$, we rebuild the largest level, T_L , as in standard hierarchical ORAM. After its first rebuild, T_L is also always full.

4.2 The Construction

We are now ready for the full description of our ORAM.

Construction 4.2: Oblivious RAM

- **Secret state:** Levels $T_0, T_1, \dots, T_L$ which are HT tables (initialized to empty structures), counters $\text{Load}_0, \dots, \text{Load}_L$ that are initialized to 0. Also define $\text{curr} = \perp$.
 - **Access**(op, addr, data) where $\text{op} \in \{\text{read}, \text{write}\}$, $\text{addr} \in [N]$, $\text{data} \in \{0, 1\}^w$:
 1. Initialize $\text{found} := \text{false}$, $\text{data}^* := \perp$.
 2. For each $i \in \{0, 1, \dots, L\}$ in increasing order:
 - (a) If $\text{found} = \text{false}$:
 - i. If $\text{Load}_i > 0$, run $\text{fetchd} := T_i.\text{Lookup}(\text{addr})$.
 - ii. If $\text{fetchd} \neq \perp$, then let $\text{data}^* := \text{fetchd}$, $\text{found} := \text{true}$.
 - (b) Else, if $\text{Load}_i > 0$, then perform $T_i.\text{Lookup}(\perp)$.
 3. If $\text{found} = \text{false}$, it is the first time addr is being accessed, so we set $\text{data}^* := 0$.
 4. Let $(k, v) := (\text{addr}, \text{data}^*)$ if this is a **read** operation. Otherwise, this is a **write** operation, let $(k, v) := (\text{addr}, \text{data})$. Set $\text{curr} := (k, v)$.
 5. Rebuild:
 - (a) Let $j < L$ be the smallest index such that $\text{Load}_j < q - 1$. If such j exists:
 - i. Let $\mathcal{U} = \text{curr} \| T_0.\text{Extract}() \| \dots \| T_{j-1}.\text{Extract}() \| T_j.\text{Extract}()$.
 - ii. Run $T_j.\text{Build}(\mathcal{U})$.
 - iii. Reset $\{\text{curr}, \dots, T_{j-1}\}$ to empty structure. Increment Load_j by 1, and set the flags $\text{Load}_0, \dots, \text{Load}_{j-1}$ to 0.
 - (b) Else: (i.e., rebuilding all elements into T_L)
 - i. Let $\mathcal{U} = \text{curr} \| T_0.\text{Extract}() \| \dots \| T_{L-1}.\text{Extract}() \| T_L.\text{Extract}()$.
 - ii. Let $\mathcal{U}' = \text{Compaction}(\mathcal{U})$ that moves all real elements to $\mathcal{U}'$ of size N (i.e., N/B memory blocks).
 - iii. Run $T_L.\text{Build}(\mathcal{U}')$.
 - iv. Reset $\{\text{curr}, \dots, T_{L-1}\}$ to empty structure. Set the flags $\text{Load}_0, \dots, \text{Load}_{L-1} = 0$ and $\text{Load}_L = q - 1$.
 - **Output:** Return data^*
-

Theorem 4.3. *Let $N \in \mathbb{N}$ be the number of words in the logical memory, where each memory word is $w = \Omega(\log N)$ bits. Let $B = \Omega(\log^{1+\epsilon} N)$ be the ratio between the server (physical) and the client (logical) word sizes for any constant $\epsilon > 0$, and let $m = \Omega(Bw)$ be the number of bits in the client's local space. Construction 4.2 obviously implements Functionality 4.1. For some parameter $q \in [\sqrt{B}, (B/\log N) + 1]$:*

- The amortized I/O overhead of the construction is $O\left(\frac{\log(Nw/m)}{\log q}\right)$;
- The failure probability is $O\left(\frac{\log(Nw/m)}{\log q}\right) \cdot e^{-B/6} \cdot T$ over any sequence of T logical accesses.

While Theorem 4.3 is asymptotically tight when $B = \log^{1+\epsilon} N$, the constant in big-O is proportional to $1/\epsilon$, and the failure probability $e^{-B/6}$ is often insufficient for practical N . Thus, we recommend to apply the above theorem when $B \geq \log^2 N$.

Corollary 4.4. *Let N be the total number of cells in the logical memory, and let $w \geq \log N$. Suppose that $B = b/w \geq \log^2 N$. Then, Construction 4.2 obviously implements Functionality 4.1 and takes I/O overhead $4 \cdot \lfloor \frac{\log N}{\log q} \rfloor + 2 + o(1)$, where $q = B/\log N + 1$, where the failure probability is $O\left(\frac{\log N}{\log q}\right) \cdot e^{-B/6} \cdot T$ over T logical accesses.*

Further suppose that $B = \omega(\log^4 N)$, then the I/O overhead is reduced to $2 \cdot \lfloor \frac{\log N}{\log q} \rfloor + 4 + o(1)$, where $q = B/\log^2 N$.

We prove these theorem and corollary in Appendix C.2. We also review how to extend the construction to larger local memories to fully match the lower bound of [KL21]. We also show how to remove the assumption of perfect powers.

5 Implementation and Evaluation

In this section, we evaluate the efficiency of our construction. First, we give some details about our implementation. We then examine how different parameters (physical block size, q , and so on) affect the performance. We also compare the efficiency of our implementation to the expected theoretical bounds. Finally, we also compare our construction to a reference implementation of Path ORAM [SS13b] and FutORAMa [AKM23].

5.1 Evaluation of Our Construction

We implemented our solution on Python 3.11 and conducted the experiments locally on a 16 GB MacBook Air M2 (A2941). We have the following modes for the implementation:

1. **Full mode:** This mode allocates the entire memory (dynamically), and each access to the ORAM also reads/writes from that memory. Among other things, this allows us to verify the correctness of the scheme by comparing what element is retrieved in each memory cell versus what is expected to reside there.
2. **Simulation mode:** This mode does not allocate the memory but runs the entire ORAM construction. Whenever accessing the external memory, we pretend to read/write, but there is no actual storage. We use this mode for efficiency reasons, which allows us to operate on larger memory sizes.
3. **Counting mode:** Here, we do not run the entire ORAM construction. We *tightly* calculate how often each level is being accessed and count the exact I/O without performing it. This is in contrast to the theoretical analysis, which also introduces some upper bounds and does not consider internal memory.

Conducting the tests. Each test we report was conducted as follows. We initialize the ORAM so that the last level is full. Then, we run $2N$ random accesses on the ORAM. This includes choosing

whether to read or write at random, the address being accessed at random, and the data (in terms of writing) also being random.

Our ORAM can work with an empty structure and add the elements adaptively (in fact, this is the way Functionality 4.1 is implemented). Still, we conducted the experiments when the structure was full to avoid skewing the results due to the initialization. Moreover, it allows a more fair comparison to Path ORAM.

Error probability. The error probability of our scheme is bounded by $T \cdot \epsilon$, where $\epsilon = \frac{\log N/m}{\log B} \cdot e^{-B/6}$ (see Theorem 4.3). We chose parameters such that $\epsilon < 2^{-128}$, (and in most cases, the error is way smaller). This corresponds to “classical” security parameter of 128 bits, e.g., breaking symmetric encryption with key of size 128, where an adversary that runs in time 2^ρ can succeed with probability $2^{\rho-128}$.

Table 2: Comparing the three modes. We first compare our expected number of accesses according to the three modes. In this experiment, we fixed $N = 2^{16}$, $w = 32$ bits, $q = \sqrt{B}$, and internal memory of 8 server blocks (4 are needed for the ORAM to work, and 4 that holds levels T_0 and T_1). We use different physical block sizes b and thus B and q . We compare the (average) number of physical blocks being accessed in the full mode, simulation mode, and counting mode. We also provide two ratios: The ratio between the I/O overhead and $\log N / \log \log N$, and the ratio between the I/O overhead and $\log_q N$. Finally, we also report the error probability for those sets of parameters. See Table 2.

We see perfect consistency between the different modes of our ORAM. This enables us to scale the experiments to much larger memory sizes. The experiment also shows that when B gets larger, the I/O overhead gets smaller, as expected.

Table 3: Increasing local memory. We now investigate how increasing the internal memory reduces I/O overhead. When we have a larger internal memory, we can store the top levels at the client. Recall that our construction matches the lower bound of Komargodski and Lin [KL21] and achieves $O(\log(Nw/m)/(1 + \log B))$ I/O overhead, and we now examine how m affects the overhead.

We run with $N = 2^{30}$, $w = 32$ (and so total memory of 2^{35} bits $\approx 4\text{GB}$). We use $b = w^3 = 4KB$, $B = w^2 = 1024$, $q = 32$, and vary the local memory size m . Previously, the experiments ran with 8 server blocks on the client ($= 32KB$), comprising 4 blocks for storing the top levels, and 4 blocks for work as bin packing operates on 4 blocks at a time. Here, we allow a larger local memory.

Table 4: I/O as a function of N . Next, we fix $w = 32$ and $b = 4KB$ (and thus $B = 1024$), which implies that the error probability is bounded by 2^{-210} . We use $q = \sqrt{B}$. We run experiments with various values of N , and see how the I/O overhead changes as N grows. This experiment was conducted in the simulation mode. We also compare it to our theoretical analysis, Eq. (3) in the Appendix. The theoretical analysis always upper bounds the number of accesses that took place in the simulation, and it becomes tighter as N grows. See Table 4.

Table 5: I/O as a function of q . We show the average number of physical accesses as a function of q , the expansion factor in the ORAM. We take $N = 2^{16}$ and use the full mode. We take $w = 32$, $b = w^3 = 4KB$, and so $B = 1024$ (and the error is 2^{-227}). We run on values of $q \in [\sqrt{B}, \dots, B/\log N + 1] = [32, 64]$ in intervals of 4. The experiment was conducted in full mode. Generally, the number of accesses as a function of q depends on the exact rounding of N and $q^i(q-1)$ since we always round up the size of a level to the next power of 2. Moreover, we see how the overhead decreases between 44 and 48 because the number of levels decreases.

5.2 Comparison to Path ORAM and FutORAMa

We compare our scheme to Path ORAM [SS13b] and FutORAMa [AKM23].

- Both Path ORAM and FutORAMa were not designed for the asymmetric case, where the logical word size and the physical block size are not the same.⁴ Thus, in this case, one can “pack” N elements in N/B physical blocks and use the schemes for $N' = N/B$ elements with b as the block size.
- We use the recursive version of the reference implementation of Path ORAM.⁵ Since each physical block can store $B = b/w$ elements, the primary data tree contains N/B leaves, effectively packing N elements into N/B blocks. Additionally, every subsequent position map tree in the recursive structure operates on a smaller address space by a $1/B'$ factor, where $B' = b/\log n$, where n is the number of blocks in the current step of the recursion. This is because the size of each entry in the position map is $\log n$. Here, B' serves as the multiplicative factor for $\log n$, corresponding to the χ parameter described in [SS13b]. We use bucket size $Z = 4$, which is the number of blocks each node holds.

Various block sizes. We compare our scheme with Path ORAM and FutORAMa when using different block sizes b . We set the memory size to be $1\text{TB} = 2^{43}$ bits and $w = 64$.

Table 6: A comparison between our ORAM and Path ORAM for small local memory (recall that this is a regime where FutORAMa cannot work). Both constructions use the exact same local memory. Our improvement is in the range of $10\times$ to $30\times$.

Table 7: We compare our scheme to Path ORAM and to FutORAMa. To accommodate FutORAMa, we use a large local memory. Specifically, we use a local memory size of 131,220 physical blocks as FutORAMa requires. We also modify our construction and Path ORAM to use the same size of local memory. For Path ORAM, we terminate the recursion earlier. Our construction cannot handle small block sizes and so we do not report results smaller than 4KB. As observed in [AKM23], FutORAMa has a smaller overhead than Path ORAM for small blocks (a factor of ≈ 6), but the difference between the two schemes becomes much less significant with larger blocks. Our scheme, on the other hand, becomes much better for larger and larger blocks, with improvements in the realm of $6\times$ to $30\times$.

It is not our goal to optimize any efficiency metric across *varying* block sizes since our ORAM aims to minimize I/O overhead for the *given* block size. Indeed, in an ORAM deployment, if the block size could be chosen arbitrarily and the goal was to minimize the bandwidth, then choosing a small block size would be better, e.g., using FutORAMa and $b = 32\text{B}$ as per Table 7.

6 Conclusions

This work introduces MegaBlocks, a novel hierarchical ORAM construction that breaks the long-standing logarithmic I/O overhead barrier by achieving $O(\log N / \log \log N)$ overhead in the asym-

⁴In Path ORAM [SS13b], a “block” referred to both the logical word and the physical block as they were the same size. However, in our discussion, a block is always physical.

⁵There are several known optimizations for Path ORAM (e.g., [DvDF⁺16, CS17] or applying van Emde Boas layout [vEB75]) but we are not aware of an end-to-end implementation of an optimized version. Indeed, Ring ORAM is claimed to be 2-4x better than Path ORAM [RFK⁺15, Table 1], but we could not find a reference implementation. Additionally, we compare ORAMs in the *passive-server* setting, which excludes schemes that utilize server computation, such as a variant of Ring ORAM and Onion ORAM [DvDF⁺16].

metric setting where *physical blocks are substantially larger than logical words*. The two main technical contributions are a data management technique and a dynamic packing technique utilizing the asymmetric setting.

Remarkably, MegaBlocks not only matches the theoretical lower bound but also achieves exceptional concrete performance, with leading constants as 4 (theoretical bound), and even smaller in practice (⌈1 in some settings). Prior to this, such schemes required substantial client-side memory or suffered from sub-optimal overhead. Our construction operates with minimal local storage and enjoys optimal asymptotics, making it highly practical across a broader range of real-world systems, when the physical block size is large.

Beyond the theoretical contributions, we provide an implementation and extensive evaluation demonstrating that MegaBlocks significantly outperforms leading ORAM systems like Path ORAM and FutORAMa, achieving up to $\times 30$ improvement in bandwidth when the physical block size is 64KB. The simplicity of the design further enhances its appeal, avoiding the complexity and heavy engineering burden often associated with hierarchical ORAMs. With these advances, we establish that sub-logarithmic I/O overhead is not only achievable but also practical, opening the door for secure, efficient, and scalable oblivious computation in modern architectures.

b	B	I/O Overhead			Ratio1	Ratio2	Error
		Full	Sim	Cnt			
4KB	1024	5.21	5.21	5.21	1.13	1.63	2^{-227}
8KB	2048	3.78	3.78	3.78	0.82	1.31	2^{-473}
16KB	4096	2.40	2.40	2.40	0.52	0.90	2^{-965}
32KB	8192	2.08	2.08	2.08	0.45	0.85	$< 2^{-1000}$
64KB	16384	2.01	2.01	2.01	0.44	0.88	$< 2^{-1000}$
128KB	32768	2	2	2	0.43	0.88	$< 2^{-1000}$

Table 2: Fixed $N = 2^{16}$, logical word size $w = 32$, expansion factor $q = \sqrt{B}$, local memory is 8 blocks ($8 \cdot b$), and we vary over b . Full is Full mode; Sim is Simulation mode; Cnt is Counting mode. Ratio1 is the ratio between I/O overhead and $\log N / \log \log N$. Ratio2 is the ratio between I/O overhead and $\log_q N$.

Local memory (blocks)	Local memory (bits)	I/O overhead
2	8KB	21.1
4	16KB	18.91
64	256KB	15.97
2048	8MB	12.22
65536	256MB	7.68

Table 3: Average number of physical blocks accessed as a function of the local memory size. Here we fix a total logical memory of 2^{35} bits $\approx 4\text{GB}$ ($N = 2^{30}$, word size $w = 32$), fix physical memory of $b = w^3 = 4\text{KB}$, and fix parameter $q = 32$, but vary the local memory size.

I/O Overhead			
N	Simulation	Theory, Eq. (3)	Ratio
2^{16}	12.24	15.52	$\times 0.79$
2^{20}	15.92	18.72	$\times 0.85$
2^{24}	19.85	21.92	$\times 0.90$
2^{28}	22.83	25.12	$\times 0.91$
2^{32}	27.13	28.32	$\times 0.96$

Table 4: I/O as a function of number of cells (N). We fix $w = 32$, $b = 4\text{KB}$ (and thus $B = 1024$), $q = \sqrt{B} = 32$, and we store only a single block at the client (+4 blocks for work) – i.e., local memory is 20KB. I/O simulation is the average number of physical blocks accessed for every logical access. I/O theory is the number of physical blocks according to Eq. (3). Ratio is the ratio between the theoretical analysis and the simulation.

q	32	36	40	44	48	52	56	60	64
I/O	5.21	5.42	5.54	6.45	5.72	5.19	4.72	4.39	4.12

Table 5: I/O as a function of expansion factor (q). We fix $w = 32$, $b = w^3 = 4\text{KB}$, $N = 2^{16}$. The local memory is 10 server blocks (40KB).

Block size (b)	Local memory size ($400 \cdot b$)	Our ORAM		Path ORAM [SS13b]	
		I/O overhead	Bandwidth	I/O overhead	Bandwidth
4KB	1.6MB	37	147KB	384	1.5MB
8KB	3.25MB	24	189KB	360	2.8MB
16KB	6.25MB	18	288KB	336	5.25MB
32KB	12.5MB	13	411KB	312	9.75MB
64KB	25MB	9.3	597KB	288	18MB

Table 6: Comparing our scheme in terms of I/O and Bandwidth to Path ORAM on different block sizes. The size of the memory is 1TB. (FutORAMa is not shown as it cannot work with small local memory.)

Block size (b)	Local memory size	Our ORAM		Path ORAM [SS13b]		FutORAMa [AKM23]	
		I/O overhead	Bandwidth	I/O overhead	Bandwidth	I/O overhead	Bandwidth
32B	4MB	--	--	1600	50KB	327	10.2KB
256B	32MB	--	--	648	162KB	266	66.5KB
512B	64MB	--	--	456	228KB	246	123KB
1KB	128MB	--	--	432	432KB	227	227KB
4KB	512MB	26.39	106KB	384	1.5MB	191	765KB
8KB	1GB	15.3	122KB	224	1.75MB	174	1.3MB
16KB	2GB	10.18	162KB	216	3.34MB	157	2.4MB
32KB	4GB	9.8	313KB	208	6.5MB	188	5.8MB
64KB	8GB	6.5	416KB	200	12.5MB	171	10.6MB

Table 7: Comparing our scheme in terms of I/O and Bandwidth to Path ORAM and FutORAMa on various block sizes. All schemes simulate a memory of 1TB, with the same local memory size. Starting from 4KB, the error probability of our scheme is $< 2^{-128}$. Our scheme cannot accommodate small blocks (the error is too large).

References

- [ACN⁺17] Gilad Asharov, T.-H. Hubert Chan, Kartik Nayak, Rafael Pass, Ling Ren, and Elaine Shi. Oblivious computation with data locality. *IACR Cryptology ePrint Archive*, 2017:772, 2017.
- [ACN⁺20] Gilad Asharov, T.-H. Hubert Chan, Kartik Nayak, Rafael Pass, Ling Ren, and Elaine Shi. Bucket oblivious sort: An extremely simple oblivious sort. In *3rd Symposium on Simplicity in Algorithms, SOSA 2020, Salt Lake City, UT, USA, January 6-7, 2020*, pages 8–14. SIAM, 2020.
- [AKL⁺20] Gilad Asharov, Ilan Komargodski, Wei-Kai Lin, Enoch Peserico, and Elaine Shi. Oblivious parallel tight compaction. *Cryptology ePrint Archive*, Report 2020/125, 2020. <https://eprint.iacr.org/2020/125>.
- [AKL⁺23a] Gilad Asharov, Ilan Komargodski, Wei-Kai Lin, Kartik Nayak, Enoch Peserico, and Elaine Shi. OptORAMa: Optimal Oblivious RAM. *J. ACM*, 70(1):4:1–4:70, 2023.
- [AKL⁺23b] Gilad Asharov, Ilan Komargodski, Wei-Kai Lin, Kartik Nayak, Enoch Peserico, and Elaine Shi. OptORAMa: Optimal oblivious RAM. *J. ACM*, 70(1):4:1–4:70, 2023.
- [AKLS23] Gilad Asharov, Ilan Komargodski, Wei-Kai Lin, and Elaine Shi. Oblivious RAM with worst-case logarithmic overhead. *J. Cryptol.*, 36(2):7, 2023.
- [AKM23] Gilad Asharov, Ilan Komargodski, and Yehuda Michelson. FutORAMa: A Concretely Efficient Hierarchical Oblivious RAM. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, pages 3313–3327. ACM, 2023.
- [AKS83a] Miklós Ajtai, János Komlós, and Endre Szemerédi. An $O(n \log n)$ sorting network. In *ACM STOC*, pages 1–9, 1983.

- [AKS83b] Miklós Ajtai, János Komlós, and Endre Szemerédi. An $o(n \log n)$ sorting network. In David S. Johnson, Ronald Fagin, Michael L. Fredman, David Harel, Richard M. Karp, Nancy A. Lynch, Christos H. Papadimitriou, Ronald L. Rivest, Walter L. Ruzzo, and Joel I. Seiferas, editors, *Proceedings of the 15th Annual ACM Symposium on Theory of Computing, 25-27 April, 1983, Boston, Massachusetts, USA*, pages 1–9. ACM, 1983.
- [BCP15] Elette Boyle, Kai-Min Chung, and Rafael Pass. Large-scale secure computation: Multi-party computation for (parallel) RAM programs. In *CRYPTO*, 2015.
- [BCP16] Elette Boyle, Kai-Min Chung, and Rafael Pass. Oblivious parallel RAM and applications. In *Theory of Cryptography - 13th International Conference, TCC*, pages 175–204, 2016.
- [BN16] Elette Boyle and Moni Naor. Is there an oblivious RAM lower bound? In *Proceedings of the 7th ACM Conference on Innovations in Theoretical Computer Science, ITCS*, pages 357–368, 2016.
- [BNP⁺15] Vincent Bindschaedler, Muhammad Naveed, Xiaorui Pan, XiaoFeng Wang, and Yan Huang. Practicing oblivious access on cloud storage: the gap, the fallacy, and the new way forward. In *ACM CCS*, pages 837–849, 2015.
- [CGLS17a] T.-H. Hubert Chan, Yue Guo, Wei-Kai Lin, and Elaine Shi. Oblivious hashing revisited, and applications to asymptotically efficient ORAM and OPRAM. In *ASIACRYPT*, 2017.
- [CGLS17b] T.-H. Hubert Chan, Yue Guo, Wei-Kai Lin, and Elaine Shi. Oblivious hashing revisited, and applications to asymptotically efficient ORAM and OPRAM. In Tsuyoshi Takagi and Thomas Peyrin, editors, *Advances in Cryptology - ASIACRYPT 2017 - 23rd International Conference on the Theory and Applications of Cryptology and Information Security, Hong Kong, China, December 3-7, 2017, Proceedings, Part I*, volume 10624 of *Lecture Notes in Computer Science*, pages 660–690. Springer, 2017.
- [CGLS18] T.-H. Hubert Chan, Yue Guo, Wei-Kai Lin, and Elaine Shi. Cache-oblivious and data-oblivious sorting and applications. In *SODA*, pages 2201–2220, 2018.
- [CKW17] David Cash, Alptekin Küpçü, and Daniel Wichs. Dynamic proofs of retrievability via oblivious RAM. *J. Cryptology*, 30(1):22–57, 2017.
- [CS17] T.-H. Hubert Chan and Elaine Shi. Circuit OPRAM: unifying statistically and computationally secure orams and oprams. In *Theory of Cryptography - 15th International Conference, TCC*, pages 72–107, 2017.
- [DMN11] Ivan Damgård, Sigurd Meldgaard, and Jesper Buus Nielsen. Perfectly secure oblivious RAM without random oracles. In *Theory of Cryptography - 8th Theory of Cryptography Conference, TCC*, pages 144–163, 2011.
- [DO20] Samuel Dittmer and Rafail Ostrovsky. Oblivious tight compaction in $o(n)$ time with smaller constant. In Clemente Galdi and Vladimir Kolesnikov, editors, *Security and Cryptography for Networks - 12th International Conference, SCN 2020, Amalfi, Italy*,

September 14-16, 2020, *Proceedings*, volume 12238 of *Lecture Notes in Computer Science*, pages 253–274. Springer, 2020.

- [DvDF⁺16] Srinivas Devadas, Marten van Dijk, Christopher W. Fletcher, Ling Ren, Elaine Shi, and Daniel Wichs. Onion ORAM: A constant bandwidth blowup oblivious RAM. In *Theory of Cryptography - TCC*, pages 145–174, 2016.
- [FDD12] Christopher W Fletcher, Marten van Dijk, and Srinivas Devadas. A secure processor architecture for encrypted computation on untrusted programs. In *Proceedings of the seventh ACM workshop on Scalable trusted computing*, pages 3–8. ACM, 2012.
- [FRK⁺15] Christopher W. Fletcher, Ling Ren, Albert Kwon, Marten van Dijk, and Srinivas Devadas. Freecursive ORAM: [nearly] free recursion and integrity verification for position-based oblivious RAM. In *Proceedings of the 20th International Conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS*, pages 103–116. ACM, 2015.
- [GHJR15] Craig Gentry, Shai Halevi, Charanjit Jutla, and Mariana Raykova. Private database access with HE-over-ORAM architecture. In *International Conference on Applied Cryptography and Network Security*, pages 172–191. Springer, 2015.
- [GHRW14] Craig Gentry, Shai Halevi, Mariana Raykova, and Daniel Wichs. Outsourcing private RAM computation. In *FOCS*. IEEE Computer Society, 2014.
- [GKK⁺12] S. Dov Gordon, Jonathan Katz, Vladimir Kolesnikov, Fernando Krell, Tal Malkin, Mariana Raykova, and Yevgeniy Vahlis. Secure two-party computation in sublinear (amortized) time. In *the ACM Conference on Computer and Communications Security, CCS*, pages 513–524, 2012.
- [GM11] Michael T. Goodrich and Michael Mitzenmacher. Privacy-preserving access of outsourced data via oblivious RAM simulation. In Luca Aceto, Monika Henzinger, and Jirí Sgall, editors, *Automata, Languages and Programming - 38th International Colloquium, ICALP 2011, Zurich, Switzerland, July 4-8, 2011, Proceedings, Part II*, volume 6756 of *Lecture Notes in Computer Science*, pages 576–587. Springer, 2011.
- [GO96] Oded Goldreich and Rafail Ostrovsky. Software Protection and Simulation on Oblivious RAMs. *Journal of the ACM*, 43(3):431–473, 1996.
- [Gol87] Oded Goldreich. Towards a theory of software protection and simulation by oblivious RAMs. In *Proceedings of the 19th Annual ACM Symposium on Theory of Computing*, pages 182–194. ACM, 1987.
- [Goo11] Michael T. Goodrich. Data-oblivious external-memory algorithms for the compaction, selection, and sorting of outsourced data. In *Proceedings of the 23rd Annual ACM Symposium on Parallelism in Algorithms and Architectures, SPAA*, pages 379–388, 2011.
- [JLS21] Zahra Jafargholi, Kasper Green Larsen, and Mark Simkin. Optimal oblivious priority queues. In Dániel Marx, editor, *Proceedings of the 2021 ACM-SIAM Symposium on*

Discrete Algorithms, SODA 2021, Virtual Conference, January 10 - 13, 2021, pages 2366–2383. SIAM, 2021.

- [KL21] Ilan Komargodski and Wei-Kai Lin. A Logarithmic Lower Bound for Oblivious RAM (for all parameters). In *Advances in Cryptology – CRYPTO 2021: 41st Annual International Cryptology Conference*, volume 12826 of *Lecture Notes in Computer Science*, pages 665–694, Berlin, Heidelberg, 2021. Springer.
- [KLO12a] Eyal Kushilevitz, Steve Lu, and Rafail Ostrovsky. On the (in)security of hash-based oblivious RAM and a new balancing scheme. In *Proceedings of the Twenty-Third Annual ACM-SIAM Symposium on Discrete Algorithms, SODA*, pages 143–156, 2012.
- [KLO12b] Eyal Kushilevitz, Steve Lu, and Rafail Ostrovsky. On the (in)security of hash-based oblivious RAM and a new balancing scheme. In Yuval Rabani, editor, *Proceedings of the Twenty-Third Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2012, Kyoto, Japan, January 17-19, 2012*, pages 143–156. SIAM, 2012.
- [LHM⁺15] Chang Liu, Austin Harris, Martin Maas, Michael W. Hicks, Mohit Tiwari, and Elaine Shi. Ghost rider: A hardware-software system for memory trace oblivious computation. In *ASPLOS*, pages 87–101, 2015.
- [LN18] Kasper Green Larsen and Jesper Buus Nielsen. Yes, there is an oblivious RAM lower bound! In *CRYPTO*, 2018.
- [LO13] Steve Lu and Rafail Ostrovsky. Distributed oblivious RAM for secure two-party computation. In *TCC*, pages 377–396, 2013.
- [LWN⁺15] Chang Liu, Xiao Shaun Wang, Kartik Nayak, Yan Huang, and Elaine Shi. ObliVM: A programming framework for secure computation. In *IEEE S&P*, 2015.
- [MLS⁺13a] Martin Maas, Eric Love, Emil Stefanov, Mohit Tiwari, Elaine Shi, Krste Asanovic, John Kubiawicz, and Dawn Song. PHANTOM: practical oblivious computation in a secure processor. In *ACM CCS*, pages 311–324, 2013.
- [MLS⁺13b] Martin Maas, Eric Love, Emil Stefanov, Mohit Tiwari, Elaine Shi, Krste Asanovic, John Kubiawicz, and Dawn Song. PHANTOM: practical oblivious computation in a secure processor. In *CCS*, pages 311–324, 2013.
- [mmG25] AArch64 memory management Guide. Translation granule. <https://developer.arm.com/documentation/101811/0104/Translation-granule>, 2025. Accessed April, 2025.
- [NFO24] Daniel Noble, Brett Hemenway Falk, and Rafail Ostrovsky. Metadoram: Breaking the log-overhead information theoretic barrier. *IACR Cryptol. ePrint Arch.*, page 11, 2024.
- [NFR⁺17] Kartik Nayak, Christopher W. Fletcher, Ling Ren, Nishanth Chandran, Satya V. Lokam, Elaine Shi, and Vipul Goyal. HOP: hardware makes obfuscation practical. In *NDSS*, 2017.
- [OS97a] Rafail Ostrovsky and Victor Shoup. Private information storage (extended abstract). In *STOC*, 1997.

- [OS97b] Rafail Ostrovsky and Victor Shoup. Private information storage (extended abstract). In *Proceedings of the 29th Annual ACM Symposium on Theory of Computing*, pages 294–303. ACM, 1997.
- [PPRY18] Sarvar Patel, Giuseppe Persiano, Mariana Raykova, and Kevin Yeo. Panorama: Oblivious RAM with logarithmic overhead. In Mikkel Thorup, editor, *59th IEEE Annual Symposium on Foundations of Computer Science, FOCS 2018, Paris, France, October 7-9, 2018*, pages 871–882. IEEE Computer Society, 2018.
- [PPY18] Sarvar Patel, Giuseppe Persiano, and Kevin Yeo. CacheShuffle: A Family of Oblivious Shuffles. In Ioannis Chatzigiannakis, Christos Kaklamanis, Dániel Marx, and Donald Sannella, editors, *45th International Colloquium on Automata, Languages, and Programming (ICALP 2018)*, volume 107 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 161:1–161:13, Dagstuhl, Germany, 2018. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.
- [RFK⁺15] Ling Ren, Christopher W. Fletcher, Albert Kwon, Emil Stefanov, Elaine Shi, Marten van Dijk, and Srinivas Devadas. Constants count: Practical improvements to oblivious RAM. In *24th USENIX Security Symposium, USENIX Security*, pages 415–430, 2015.
- [RFK⁺19] Ling Ren, Christopher W. Fletcher, Albert Kwon, Marten van Dijk, and Srinivas Devadas. Design and implementation of the ascend secure processor. *IEEE Trans. Dependable Secur. Comput.*, 16(2):204–216, 2019.
- [RYF⁺13] Ling Ren, Xiangyao Yu, Christopher W. Fletcher, Marten van Dijk, and Srinivas Devadas. Design space exploration and optimization of path oblivious RAM in secure processors. In *The 40th Annual International Symposium on Computer Architecture, ISCA*, pages 571–582, 2013.
- [SCSL11] Elaine Shi, T.-H. Hubert Chan, Emil Stefanov, and Mingfei Li. Oblivious RAM with $o((\log n)^3)$ worst-case cost. In *Advances in Cryptology - ASIACRYPT*, pages 197–214, 2011.
- [Shi20] Elaine Shi. Path oblivious heap: Optimal and practical oblivious priority queue. In *2020 IEEE Symposium on Security and Privacy, SP*, pages 842–858, 2020.
- [Sig22] Signal. Technology deep dive: Building a faster oram layer for enclaves. <https://signal.org/blog/building-faster-oram>, 2022. Accessed: 2025-04-03.
- [SS13a] Emil Stefanov and Elaine Shi. Oblivistore: High performance oblivious cloud storage. In *IEEE S&P*, pages 253–267, 2013.
- [SS13b] Emil Stefanov and Elaine Shi. Path ORAM: An Extremely Simple Oblivious RAM Protocol. *Proceedings of the 2013 ACM SIGSAC Conference on Computer and Communications Security*, pages 299–310, 2013.
- [SSS12] Emil Stefanov, Elaine Shi, and Dawn Xiaodong Song. Towards practical oblivious RAM. In *19th Annual Network and Distributed System Security Symposium, NDSS*, 2012.

- [SvDS⁺18] Emil Stefanov, Marten van Dijk, Elaine Shi, T.-H. Hubert Chan, Christopher W. Fletcher, Ling Ren, Xiangyao Yu, and Srinivas Devadas. Path ORAM: an extremely simple oblivious RAM protocol. *J. ACM*, 65(4):18:1–18:26, 2018.
- [vEB75] P. van Emde Boas. Preserving order in a forest in less than logarithmic time. In *16th Annual Symposium on Foundations of Computer Science (sfcs 1975)*, pages 75–84, 1975.
- [WCSH16] Ting Wang, T.-H. Hubert Chan, Elaine Shi, and Yan Huang. Circuit ORAM: On Tightness of the Goldreich-Ostrovsky Lower Bound. *ACM Transactions on Information and System Security*, 18(4):1–32, 2016.
- [WHC⁺14] Xiao Shaun Wang, Yan Huang, T.-H. Hubert Chan, Abhi Shelat, and Elaine Shi. SCORAM: oblivious RAM for secure computation. In *ACM CCS*, pages 191–202, 2014.
- [WST12] Peter Williams, Radu Sion, and Alin Tomescu. Privatefs: A parallel oblivious file system. In *ACM CCS*, 2012.
- [ZWR⁺16] Samee Zahur, Xiao Shaun Wang, Mariana Raykova, Adria Gascón, Jack Doerner, David Evans, and Jonathan Katz. Revisiting square-root ORAM: efficient random access in multi-party computation. In *IEEE S&P*, pages 218–234, 2016.

A Additional Preliminaries

A.1 Oblivious Simulation of a Non-Reactive Functionality

Let f be a potentially randomized functionality that computes the output depending only on the input. Such functionalities are stateless and called *non-reactive*. We can define an oblivious simulation of f as follows:

Definition A.1 (Oblivious simulation of a functionality). *Let f be a functionality, and let P_f be a RAM program as defined in 2. We say that P_f obviously simulates f if there is exists a PPT simulator $\mathcal{S}$ such that for every input x :*

$$\{\text{AccessPtrn}(P_f(x)), P_f(x)\} \equiv \{\mathcal{S}(|x|), f(x)\}$$

Again, $\equiv$ might refer to perfect, statistical and computational indistinguishability.

Notice that the above also defines the *oblivious simulation of a RAM program P* , as P_f is functionality equivalent to P , and P_f is oblivious. We present the definition in terms of a functionality to facilitate the security proofs. Also notice that the simulator receives just the length of the input, but not the input itself. Moreover, we require that the *joint distribution* of the machine P_f and the access pattern of $P_f(x)$ is indistinguishable from the output of the original machine P (which is defined as f), and the output of the simulator $\mathcal{S}(|x|)$. This captures obliviousness and correctness in a single definition. See the importance for that in [AKL⁺23a].

A.2 Oblivious Simulation of a Reactive Functionality

In this section, we define oblivious simulation of reactive functionalities, i.e., a computation that preserves state. We use such functionalities as, e.g., $\mathcal{F}_{\text{HT}}$ in Section 3.4 and the $\mathcal{F}_{\text{ORAM}}$ in Section 4.1.

Reactive functionality. We define a reactive functionality as a functionality that operates in stages, maintaining an *internal state between each stage*. Such a reactive functionality can be described as a sequence of functions, where each function receives the current state as input, updates it, and outputs the updated state for the next function, together with some partial output. We extend Definition A.1 to accommodate these types of functionalities. We consider a reactive functionality $\mathcal{F}$ as a reactive machine, that receives commands of the form $(\text{command}_i, \text{inp}_i)$ and produces an output out_i , while maintaining some (secret) internal state.

Implementing a reactive functionality obliviously. An implementation of $\mathcal{F}$ is a RAM program M that receives commands in the same form $(\text{command}_i, \text{inp}_i)$, and produces an output out_i . M is considered to be oblivious if there is a reactive program $\mathcal{S}$ that can produce an indistinguishable access pattern from the one of M by receiving only command_i but not inp_i . The simulator $\mathcal{S}$ is allowed to keep an internal state between commands.

In more detail, we consider a stateful adversary $\mathcal{A}$ that participates in an ideal execution or in a real one, and we require that the view of the adversary is indistinguishable in both executions. In each stage, the adversary $\mathcal{A}$ adaptively chooses the next command $(\text{command}_i, \text{inp}_i)$. See Fig. 2 for the specification of the corresponding Real/Ideal games.

Definition A.2 (Oblivious simulation of a reactive functionality). *Let $\mathcal{F}$ be a reactive functionality, and let M be a reactive RAM program. We say that M obviously simulates $\mathcal{F}$ if for any adversary $\mathcal{A}$ there is exists a PPT simulator $\mathcal{S}$ such that experiments $\text{Real}_{\mathcal{A}}^M(1^\kappa)$ and $\text{Ideal}_{\mathcal{S}}^{\mathcal{F}}(1^\kappa)$, defined in Fig. 2, are indistinguishable.*

$\text{Real}_{\mathcal{A}}^M(1^\kappa)$:	$\text{Ideal}_{\mathcal{S}}^{\mathcal{F}}(1^\kappa)$:
Let $(\text{command}_0, \text{inp}_0) \leftarrow \mathcal{A}(1^\kappa)$, let $i = 0$. Loop while $\text{command}_i \neq \perp$: $\text{out}_i, \text{Addr}_i \leftarrow M(1^\kappa, \text{command}_i, \text{inp}_i)$ $(\text{command}_{i+1}, \text{inp}_{i+1}) \leftarrow \mathcal{A}(1^\kappa, \text{out}_i, \text{Addr}_i)$ Increment i Return the output of $\mathcal{A}(1^\kappa)$	Let $(\text{command}_0, \text{inp}_0) \leftarrow \mathcal{A}(1^\kappa)$, let $i = 0$. Loop while $\text{command}_i \neq \perp$: $\text{out}_i \leftarrow \mathcal{F}(\text{command}_i, \text{inp}_i)$ $\text{Addr}_i \leftarrow \mathcal{S}(1^\kappa, \text{command}_i)$ $(\text{command}_{i+1}, \text{inp}_{i+1}) \leftarrow \mathcal{A}(1^\kappa, \text{out}_i, \text{Addr}_i)$ Increment i Return the output of $\mathcal{A}(1^\kappa)$

Figure 2: The security game of a reactive functionality $\mathcal{F}$.

Notice that the above defines statistical security in the case that $\mathcal{A}$ is unbounded, computational security in the case that $\mathcal{A}$ is PPT, and perfect security for the case that $\mathcal{A}$ is unbounded, and both experiments $(\text{Real}_{\mathcal{A}}^M(1^\kappa)$ and $\text{Ideal}_{\mathcal{S}}^{\mathcal{F}}(1^\kappa))$ are identical.

Hybrid model and composition. Using the above, we define a $\mathcal{F}$ -hybrid model, in which the machine M interacts with the memory via **read/write**-instructions as before, and in addition can also send an ideal $\mathcal{F}$ -instruction to the memory. The adversary can see that an instruction $\mathcal{F}$ was invoked; the instruction can change the data in the memory, but there is not access pattern. Note that the exact input to the function $\mathcal{F}$ is hidden from the adversary (besides, maybe, the command it operates on if $\mathcal{F}$ is reactive). For composition, if (1) $M^{\mathcal{F}}$ obviously implements a functionality $\mathcal{G}$ in the $\mathcal{F}$ -hybrid model; and that (2) a program F obviously implements $\mathcal{F}$, then a machine M^F that runs the program F instead of calling to the functionality $\mathcal{F}$, obviously simulates $\mathcal{G}$. See [AKL⁺23a] for further details.

B Supplementary Material for Bin Packing (Section 3.1)

B.1 Valid Assignment

The routing of the elements into their destination bins is possible as long as the number of real elements in each bin along the way never exceeds the capacity of the bin $- B$. That is, the algorithm does not abort in Step 3(b)iii. We now define the predicate for which the algorithm succeeds. The functionality verifies this predicate and routes the elements only if the assignment is valid.

Notation. Let $C = 2^{m+1}$ be the size of the input array (recall that we assume that C is a power of 2). For a number x , we let $[x]_2$ denote its binary representation. For assignment $\alpha_i \in [C]$, $\ell \in [0, \dots, m]$, and $k \in [0, \dots, 2^\ell - 1]$, we define the predicate $\text{MSB}([\alpha_i]_2, \ell, [k]_2)$, which obtains 1 if the first ℓ most significant bits in the binary representation of α_i equals k (in binary representation). For example,

$$\text{MSB}(01011, 1, 1) = 0 \quad , \quad \text{MSB}(01011, 2, 01) = 1 \quad ,$$

$$\text{MSB}(01011, 3, 010) = 1 \quad , \quad \text{MSB}(\perp, 3, 011) = 0 \quad .$$

We emphasize that if an element is a dummy, then the predicate always gets 0.

In each iteration, each bin $A_j^{(\ell)}$ receives elements from a super-bin of the input array of size 2^ℓ consecutive bins. That is, define the s th super bin (for $s \in \{0, \dots, n/2^\ell - 1\}$) in the ℓ th iteration:

$$X_s^{(\ell)} = \left(A_{s \cdot 2^\ell}^{(0)}, \dots, A_{s \cdot 2^\ell + 2^\ell - 1}^{(0)} \right) .$$

Then, for every $\ell \in \{0, \dots, m-1\}$ and $j \in \{0, \dots, C-1\}$, we can verify whether $A_j^{(\ell)}$ overflows, by defining:

$$\begin{aligned} s &= \lfloor j/2^\ell \rfloor, \\ k &= j \bmod 2^\ell. \end{aligned}$$

The bin $A_j^{(\ell)}$ obtains all real elements from super-bin $X_s^{(\ell)}$ in which the most significant bits in their assignment equal $[k]_2$. That is:

$$|A_j^{(\ell)}| = \left| \left\{ x = (\alpha_i, \cdot) \in X_s^{(\ell)} \mid \text{MSB}([\alpha_i]_2, \ell, [k]_2) = 1 \right\} \right|.$$

Definition B.1 (valid assignments). *An assignment $(a_i = (\alpha_i, \cdot))_{i=0}^{2n-1}$ is valid if for every $\ell \in \{0, \dots, m-1\}$, and $j \in \{0, \dots, C-1\}$, it holds that $|A_j^{(\ell)}| \leq B$.*

Random assignment is valid. We conclude this subsection by showing that when the input array is balanced (each bin is at most half full), and the assignment to the elements is chosen uniformly at random, then the assignment is valid. That is,

Claim B.2 (Random assignment is valid). *Let $X = (a_0, \dots, a_{2n-1})$ be such that:*

1. *The blocks*

$$\begin{array}{c|c} A_0^{(0)} & \dots & A_{C-1}^{(0)} \\ (a_0, \dots, a_{B-1}) & \dots & (a_{2n-B}, \dots, a_{2n-1}) \end{array}$$

are half full, namely, each $A_j^{(0)}$ contains at most $B/2$ real elements.

2. *Assign to each real a_i the assignment $(\alpha_i, \cdot)$ for a random $\alpha_i \leftarrow [C]$ (recall that an element $a_i = (\perp, \cdot)$ is dummy).*

Then, the resulting assignment is valid (satisfies Definition B.1) with probability $1 - C \cdot \log C \cdot e^{-B/6}$.

Proof. Consider a bin $A_j^{(\ell)}$ at level ℓ , and let $s = \lfloor j/2^\ell \rfloor$ and $k = j \bmod 2^\ell$. This bin gets elements from the super-bin $X_s^{(\ell)}$. Since each one of the bins in level 0 is half-full, the super-bin $X_s^{(\ell)}$, containing 2^ℓ regular bins (i.e., a total of $2^\ell \cdot B$ elements), is also half-full. Namely, it contains at most $2^\ell \cdot B/2$ elements.

The assignments of elements are distributed uniformly and independently at random in $[C]$. The probability of an assignment to have its ℓ MSB equals k is exactly $2^{-\ell}$. Therefore, overall, the expected load of real elements reaching bin $A_j^{(i)}$ is exactly $2^\ell \cdot B/2 \cdot 2^{-\ell} = B/2$. We bound the probability for overflow, using the Chernoff bound. We mark the load of a bin $A_j^{(i)}$ as $|A_j^{(i)}|$. According to Chernoff, the probability for $|A_j^{(i)}|$ to be at least $\frac{B}{2}$ -far from its expected value is:

$$\Pr \left[|A_j^{(i)}| \geq B \right] = \Pr \left[|A_j^{(i)}| \geq 2 \cdot E[|A_j^{(i)}|] \right] \leq e^{-\frac{B}{6}}.$$

Since there are $\log C$ levels and each level contains C bins, The probability for overflow across all levels and all bins according to union bound is bounded by $C \log C \cdot e^{-\frac{B}{6}}$. $\square$

Theorem B.3 (Theorem 3.3, restated). *Construction 3.2 perfectly obviously implements Functionality 3.1 for every input X with valid assignment (see Definition B.1), and it accesses $2C \log C$ number of blocks, where $C = 2n/B$.*

Proof. Assuming that the assignment is valid as in Definition B.1, the event abort in Step 3(b)iii in Construction 3.2 never occurs. The access pattern is then deterministic. Moreover, from inspection, all real elements reside at the end of the protocol in their destination bins, just as the output of the functionality. Moreover, since the access pattern is deterministic and is the same for every (valid) input, it can be simulated trivially. The efficiency follows because there are $\log C$ iterations, each iteration accesses $2C$ blocks. $\square$

C Deferred Proofs

C.1 Proof of Theorem 3.6

Theorem C.1 (Theorem 3.6, restated). *Assuming security of PRF, construction HT (Construction 3.5) implements $\mathcal{F}_{\text{HT}}$ (Functionality 3.4). For an input X of size n elements, each of size w bits:*

- **Build(X)** requires accessing $2C \log C + 1.5C$ memory blocks.
- **Lookup(k)** requires accessing 2 memory blocks (read and write of the same block).
- **Extract()** requires accessing $2C \log C + 1.5C$ memory blocks.

where C is the closest power of 2 that upper bounds $2n/B$. When modeling the PRF as a random function, the error probability of the construction is $2C \log C \cdot e^{-B/6}$.

Proof. We first show security and then discuss the efficiency.

Security. We show a simulator for HT:

The simulator $\mathcal{S}$. The adversary iteratively outputs a command and input, and receives back (in the ideal) the invocation of the simulator on the command and the output of the functionality. We define the output of the simulator on each one of the possible commands:

- When the simulator receives the command **Build($|X|$)**:
The simulator returns the deterministic access pattern of the **Build()** algorithm on the appropriate size. Note that this is just arranging the input and providing a deterministic access pattern.
- When the simulator receives the command **Lookup**: The simulator chooses a random $\alpha \leftarrow \{0, \dots, C-1\}$ and simulates an access to Bin_α .
- When the simulator receives the command **Extract()**: The simulator returns the deterministic access pattern of the **Extract()** algorithm, which is just a reverse route of the access pattern of **Build**.

We now show that the view of the adversary in the real world is indistinguishable from its view in the ideal world, according to the security definition of a simulation of reactive functionality, see Definition A.2. Assume that the adversary is valid and never make a recurrent query. We show that via a sequence of hybrid experiments:

- **Hyb₀**: This is the real execution. The adversary $\mathcal{A}$ issues commands $\text{HT.Build}(X)$ for X of its choice, and issues a sequence of commands $\text{HT.Lookup}(x_1), \dots, \text{HT.Lookup}(x_n)$ adaptively under the restriction that there is no recurrent non-dummy query. The last command is $\text{HT.Extract}()$. With each command, we execute construction 3.5 according to the relevant command, and give the adversary the corresponding access pattern and output of the command (only **Lookup** has an output). Note that the secret state is hidden from the adversary's view. Notice that this is a real execution as defined in Section A.2.
- **Hyb₁**: This is the same as the previous hybrid, while instead of using $\text{PRF}_k(\cdot)$ we use a random function $\mathcal{O}(\cdot)$. Namely, in Step 3 of **HT.Build**, instead of computing $\alpha_i = \text{PRF}_{\text{sk}}(k_i)$ we use $\alpha_i = \mathcal{O}(k_i)$. Likewise, in Step 3 of **HT.Lookup**, instead of computing $\alpha = \text{PRF}_{\text{sk}}(k)$, we use $\alpha_i = \mathcal{O}(k_i)$. The output of this hybrid is the access pattern, together with the outputs as determined by the construction.
- **Hyb₂**: This is like the previous hybrid, where with each command with the exact same input, the experiment also invokes $\mathcal{F}_{\text{HT}}$ on the corresponding command, and the simulator. Nevertheless, the adversary just sees the access pattern and output from the construction (from the real). In case the execution of **HT.Build** aborts due to bin-packing, this experiment aborts with the access pattern of **HT.Build** as well.
- **Hyb₃**: This is as the previous execution, while instead of giving the adversary $\mathcal{A}$ the access pattern and the output of the **HT.Extract()** execution from the real, we give it the output from the functionality $\mathcal{F}_{\text{HT.Extract}}$ and the access pattern from the simulation.
- **Hyb₄**: This is as the previous execution, while instead of giving the adversary $\mathcal{A}$ the access pattern and the output of the **HT.Lookup** executions from the real, we give it the output of functionality $\mathcal{F}_{\text{HT.Lookup}}$ and the access pattern from the simulation.
- **Hyb₅**: This is as the previous execution, while instead of giving the adversary $\mathcal{A}$ the access pattern and the output of the **HT.Build**(X) execution from the real, we give it the output from the functionality $\mathcal{F}_{\text{HT.Build}}$, which has no output, and the access pattern from the simulation. In this hybrid, the **HT** is never used, and this experiment is identical to the above simulator $\mathcal{S}$ (as needed in Definition A.2).

We show that each consecutive hybrids is indistinguishable.

Hyb₀ and Hyb₁ are computationally-indistinguishable assuming the security of the PRF: If there exists a distinguisher between the two hybrids, one can construct a distinguisher between PRF and a truly random function.

Hyb₁ and Hyb₂ are identical: This is just a systematic change; the view of the adversary is exactly the same.

Hyb₂ and Hyb₃ are identical: The access pattern of **HT.Extract()** algorithm is deterministic and identical to the simulated accesses. Moreover, for both **HT** and $\mathcal{F}_{\text{HT}}$, the output is just the input array X , while all elements that were accessed during the lookup phase are replaced with dummies, which is identical.

Hyb₃ and Hyb₄ are statistically-close with $C \cdot \log C \cdot e^{-B/6}$: If **HT.Build** aborts, both experiments perform identical prior to the abort. Thus, it suffices to show the probability conditioned on the good event without abort. The real **HT** and the functionality $\mathcal{F}_{\text{HT}}$ answers each **Lookup**(k_i) command identically when conditioned on the good event.

Regarding the access pattern, recall that the adversary never performs the same $\text{Lookup}(k)$ command twice. Also recall that we conditioned on the good event, and thus, the assignment of $\text{HT.Build}(X)$ must be valid (in both hybrids). The difference is in the lookups. In Hyb_3 , each HT lookup results in accessing the bin α , where α is $\mathcal{O}(k)$ if k is a real key, or α is sampled uniformly random from $[C]$. In Hyb_4 , the simulated access is always sampled uniformly random from $[C]$. Observe that the difference is that in Hyb_3 , the accesses depend on the good event and *must be valid*, while the simulated accesses are independent in Hyb_4 . Particularly, for any lookup sequence for real keys $k_0, k_1, \dots, k_t$ such that $k_i \in X$, if the simulated accesses $\alpha_0, \dots, \alpha_t$ in Hyb_4 is a valid assignment, then the simulated accesses is identical to the accesses of HT.Lookup . By Claim B.2, the random simulation is valid except for probability $C \cdot \log C \cdot e^{-B/6}$.

Hyb_4 and Hyb_5 are statistically-close with $C \cdot \log C \cdot e^{-B/6}$: The $\text{HT.Build}(X)$ algorithm and $\mathcal{F}_{\text{HT}}.\text{Build}(X)$ functionality have no output, so clearly, this does not change the adversary's view. The access pattern of $\text{HT.Build}(X)$ is deterministic, as it consists of some linear scan and then an invocation of bin packing (Construction 3.2). The only difference is that in Hyb_4 , the $\text{HT.Build}(X)$ algorithm might fail due to invalid assignment, while such a behavior does not occur in the simulated accesses of Hyb_5 . However, the assignment of each element into its destination bin is random, and each bin is at most half full (this also holds for the last $C - 2n/B$ last bins where the load is 0). Therefore, the random assignment is valid except for probability $C \cdot \log C \cdot e^{-B/6}$ by Claim B.2. Hence, the difference between Hyb_4 and Hyb_5 is statistically-close.

Efficiency. We analyze the efficiency of each command:

- $\text{HT.Build}(X)$: The $\text{HT.Build}(X)$ algorithm consists of an invocation of the Functionality 3.1 which requires a total of $2C \log$ accesses. (There are C bins and $\log C$ levels, and each bin is being accessed twice – once for writing it, and then reading it for the generation of the next level). In addition, before invoking the bin-packing, we split the elements into bins (while adding dummy elements), which requires additional $1.5C$ accesses.
- $\text{HT.Lookup}(k)$: When we invoke the $\text{HT.Lookup}(k)$ algorithm, we read one memory block ($\text{PRF}_{sk}(k)$ or a random one), and write it back. Thus, the total number of memory accesses is 2.
- $\text{HT.Extract}()$: The $\text{HT.Extract}()$ algorithm is again an invocation of Functionality 3.1 which requires $2C \log C$ accesses. We then remove the dummy bins blocks that were introduced during Build, which require also $1.5C$ accesses.

This concludes the proof of Theorem 3.6. □

C.1.1 Proof for Theorem 3.9

Theorem C.2 (Theorem 3.9, restated). *Compaction (Construction 3.8) obviously implements functionality $\mathcal{F}_{\text{Compaction}}$ (Functionality 3.7) with error probability that is bounded by $2C \cdot \log C \cdot e^{-\frac{B}{6}}$, and it accesses $2C \log C + 3C + \frac{n_0}{B}$ number of memory blocks, where $C = \frac{2n}{B}$.*

Proof. We prove correctness and security in the hybrid model where Functionality 3.1 is called in step 4. The simulator $\mathcal{S}$ takes input (n, n_0) , and performs the following:

1. Let X be an array of n elements such that the front n_0 elements are real and the remaining are dummy.
2. Execute $\text{Compaction}_{n, n_0}(X)$ and then output the accesses of the execution, including the invocation of $\mathcal{F}_{\text{BinPack}}$.

We want to show that for any valid X specified in $\mathcal{F}_{\text{Compaction}}$, it holds that the following two distributions are statistically close:

$$\begin{aligned} \text{Real: } & (\text{AccessPtrn}(\text{Compaction}_{n,n_0}(X)), \text{Compaction}_{n,n_0}(X)) . \\ \text{Ideal: } & (\mathcal{S}(n, n_0), \mathcal{F}_{\text{Compaction}}(X)) . \end{aligned}$$

We consider a hybrid construction Hyb_1 that is similar to Construction 3.8. The difference is that, in Step 4, Hyb_1 moves items according to *any* assignment without the bin size constraint in $\mathcal{F}_{\text{BinPack}}$, and thus Hyb_1 may use some bins Y_i of a size greater than B . Moreover, in the access pattern of Hyb_1 , even when $|Y_i| > B$, any access to Y_i still yields the same location. For any valid X , let $(\text{Hyb}_1^P, \text{Hyb}_1^A)$ be the output of $\text{Hyb}_1(X)$, where Hyb_1^P is the access pattern and Hyb_1^A is the array.

By Claim B.2, it holds that $\text{Hyb}_1(X)$ is ε -close to the Real experiment, where $\varepsilon = \frac{2n}{B} \cdot \log \frac{2n}{B} \cdot e^{-\frac{B}{6}}$. Also, we argue that the access pattern Hyb_1^P is independent of the array Hyb_1^A by a coupling argument. For any pair (P, A) and for any A' such that A' is obtained from A by swapping two elements, if $\text{Hyb}_1(X) = (P, A)$ using a random tape ρ , then we have $\text{Hyb}_1(X) = (P, A')$ using the random tape ρ' such that differs from ρ only in the bin assigned in Step 2 and the shuffling in Step 6. It follows that $\Pr[\text{Hyb}_1(X) = (P, A)] = \Pr[\text{Hyb}_1(X) = (P, A')]$ for all X, P, A, A' , and thus the independence follows. To show Hyb_1 and the Ideal experiment is close, by the independent marginal distributions, it suffices to show the marginals are coordinate-wise close. We have that $\text{Hyb}_1^A(X)$ and $\mathcal{F}_{\text{Compaction}}(X)$ are identical since they are both a uniform shuffle of the real elements. Also, $\text{Hyb}_1^P(X)$ and $\mathcal{S}(n, n_0)$ are identical except when $\mathcal{S}$ sampled an invalid assignment in Step 2. By Claim B.2, the assignment is valid except for ε probability. This concludes that the Real and Ideal experiments are 2ε -close statistically.

We analyze the efficiency below. The cost is dominated by calling bin packing on C groups, which takes $2C \log C = \frac{2n}{B} \log \frac{2n}{B}$ accesses by Theorem 3.3. The other minor terms are counted next. Notice that whenever there are multiple consecutive accesses to the same memory block, we coalesce the accesses into one physical access. For instance, we coalesce Step 2 and 3 by assigning random bins while moving elements from X to X' . With that, the first phase takes $2n/B$ accesses to count the number n_r and assign random bins and C accesses to prepare X' . The second phase takes C accesses to read and shuffle elements in Y and n_0/B accesses to write A . Hence, the total number of physical accesses is $2C \log C + 3C + \frac{n_0}{B}$. $\square$

C.2 Proof of Theorem 4.3

Theorem C.3 (Theorem 4.3, restated). *Let $N \in \mathbb{N}$ be the number of words in the logical memory, where each memory word is $w = \Omega(\log N)$ bits. Let $B = \Omega(\log^{1+\epsilon} N)$ be the ratio between the server (physical) and the client (logical) word sizes for any constant $\epsilon > 0$, and let $m = \Omega(Bw)$ be the number of bits in the client's local space. Construction 4.2 obviously implements Functionality 4.1, the amortized I/O overhead of the construction is $O\left(\frac{\log(Nw/m)}{\log q}\right)$ for some parameter $q \in [\sqrt{B}, (B/\log N) + 1]$, and the failure probability is $O\left(\frac{\log(Nw/m)}{\log q}\right) \cdot e^{-B/6} \cdot T$ over any sequence of T logical accesses.*

Proof. We prove the security, supposing the I/O overhead holds.

Correctness and security. The ORAM functionality (Functionality 4.1) is a reactive functionality. Hence, we will use Definition A.2 to prove security. In the hybrid model, all underlying

primitives, **HT** and **Compaction**, are instantiated using ideal functionalities, $\mathcal{F}_{\text{HT}}$ and $\mathcal{F}_{\text{Compaction}}$. Moreover, all operations of the functionalities, such as invocations of **Extract** and **Build**, are performed according to a schedule determined by the number of **Access** performed and is publicly known to the adversary. In the hybrid model, the simulation is straightforward as long as the invocation of $\mathcal{F}_{\text{HT}}$ is non-recurrent, which is verified by inspection as follows. Suppose that an address **addr** was **Accessed** in some level T_i at time t . The next time to rebuild T_i would be $\bar{t} = q_i \cdot \lceil t/q^i \rceil$. If the next **Access** to the same **addr** happens at time t' such that $t < t' \leq \bar{t}$, then **addr** is in and will be found in levels $T_0, T_1, \dots$, or T_{i-1} . Otherwise, $t' \geq \bar{t}$, then T_i is completely rebuilt. Therefore, the requirement of non-recurrent lookup is maintained in both cases. We conclude that the simulator is perfect in the $(\mathcal{F}_{\text{HT}}, \mathcal{F}_{\text{Compaction}})$ hybrid model.

When instantiating the functionalities by **HT** and **Compaction**, the total number of invocations is at most $O(T \cdot \log_q N)$. By Theorem 3.9 and Theorem 3.6, the simulation error of each invocation is $\frac{2n}{B} \cdot \log \frac{2n}{B} \cdot e^{-B/6}$ for some $n \leq N$. Because $B = \Omega(\log^2 N)$, the total error is $T \cdot \text{negl}(N)$ for a negligible function negl .

Efficiency. Recall that we assumed N is a perfect power of q . Also, $L = \log_q N$ and T_L stores N elements. For readability, we further assume that $B \geq \log^2 N$ and $m = \Theta(B)$. The assumptions will be removed later. Since $q \in (\sqrt{B}, (B/\log N) + 1]$, we have $q < B \leq q^2$. We have the following:

- **Lookup:** In each access, we perform **Lookup** on each level, which requires accessing 2 blocks (see Theorem 3.6). There are $L + 1$ levels, and thus a total of at most $2(L + 1)$ physical accesses.
- **Rebuild:** We count the cost of each level i , which includes **Extract** and **Build**. For every q^i accesses, $T_i.\text{Extract}$ and $T_i.\text{Build}$ are performed on $\text{Load}_i \cdot q^i$ elements (whereas the cost of extracting and building other levels is counted toward other i'). The cost is 2 for T_0 because $|T_0| = O(q) \leq B$. For $i = 1$, the number of elements is $\text{Load}_1 \cdot q$, which is less than B when Load_1 is small, and is greater than B when Load_1 is close to $q - 1$. Hence, the cost is $\max(2, 4 \cdot \frac{2\text{Load}_1 \cdot q}{B} \cdot \log \frac{2\text{Load}_1 \cdot q}{B})$ per q accesses by Theorem 3.6. Over a period of q^2 logical accesses, the cost is bounded by

$$2q + 4 \cdot \frac{2(q-1)q}{B} \log \frac{2(q-1)q}{B}.$$

For $i > 1$, we have $|T_i| \geq q^2 > B$. By Theorem 3.6, the cost is $4 \cdot \frac{2\text{Load}_i \cdot q^i}{B} \cdot \log \frac{2\text{Load}_i \cdot q^i}{B}$. Since the Load_i goes from 0 to $q - 1$ in a period of q^{i+1} accesses, the per-period cost is at most

$$\sum_{j=0}^{q-1} 4 \cdot \frac{2j \cdot q^i}{B} \cdot \log \frac{2j \cdot q^i}{B} \leq \frac{4(q-1) \cdot q^{i+1}}{B} \cdot \log \frac{2q^{i+1}}{B},$$

where $2j \cdot q^i/B \leq 2q^{i+1}/B$. Over $T \geq N$ accesses and amortizing over L levels, the number of accesses the rebuild (and extract) introduce is:

$$\begin{aligned} & 2 + \frac{1}{T} \cdot \frac{T}{q^2} \cdot 2q + \frac{1}{T} \cdot \sum_{i=1}^{L-1} \frac{T}{q^{i+1}} \cdot \frac{4(q-1) \cdot q^{i+1}}{B} \cdot \log \frac{2q^{i+1}}{B} \\ &= 2 + \frac{2}{q} + \sum_{i=1}^{L-1} \frac{4(q-1)}{B} \cdot (1 + (i+1) \log q + \log B) \\ &\leq 2 + \frac{2}{q} + \frac{2(L-1)(q-1) \cdot \log N}{B}, \end{aligned} \tag{1}$$

where the inequality holds since $\sum_{i=1}^{L-1} c_1 i + c_2 = (L-1)(c_1 L/2 + c_2)$, where $c_1 = \log q$ and $c_2 = 1 + \log q - \log B$, so that $c_2 \leq 0$ by $B > 2q$ and thus $c_1 L/2 + c_2 \leq (L \log q)/2 = (\log N)/2$.

- **Largest level and compaction:** Next, at level T_L , the rebuild is performed every $N = q^L$ accesses, and it extracts and compacts on $2N$ elements but build on N elements. By Theorems 3.6 and 3.9, the cost of extraction and compaction over $T \geq N$ accesses is

$$\frac{1}{T} \cdot \frac{T}{N} \cdot (4 \cdot \frac{2(2N)}{B} \log \frac{2(2N)}{B}) \leq \frac{16}{B} \cdot \log \frac{4N}{B}.$$

After the compaction (Step 5b), there are N elements to be built. The amortized cost of building N elements is $\frac{1}{T} \cdot \frac{T}{N} \cdot 2 \cdot \frac{2N}{B} \log \frac{2N}{B}$ physical accesses by Theorem 3.6. Hence, the amortized cost for the largest level T_L is

$$\frac{16}{B} \cdot \log \frac{4N}{B} + \frac{4}{B} \cdot \log \frac{2N}{B} = \frac{20}{B} \cdot \log \frac{2N}{B} + \frac{16}{B}. \quad (2)$$

Summing over the costs of lookup and rebuild (Equations (1) and (2)), the amortized number of accesses is bounded by:

$$2(L+1) + 2 + \frac{2}{q} + \frac{2(L-1)(q-1) \cdot \log N + 20}{B} + \frac{16}{B}. \quad (3)$$

By $L = \log_q N$, the claimed overhead is achieved.

Extend to mildly large blocks, $B = \Omega(\log^{1+\epsilon} N)$. Now we relax the assumption and show that our Construction 4.2 works for mildly large $B = \Omega(\log^{1+\epsilon} N)$ for any constant $\epsilon > 0$. The only modification is to choose $q = B/\log N = \Omega(\log^\epsilon N)$.

We claim that the amortized cost is still $O(\log_q N) = O(\frac{\log N}{\log \log N})$. To see why, let $B = \log^{1+\epsilon} N$ for some ϵ . Since $q = \log^\epsilon N$, the first $\ell_0 = 1/\epsilon$ levels together store less than B elements. Hence, for such small levels, the rebuild takes only 4 physical accesses, and the amortized cost over q^i logical accesses is $4/q^i = o(1)$ for each level $i \leq \ell_0$. The cost for the larger levels $i > \ell_0$ is calculated identical to Theorem 4.3. The bound $4L + 2 + o(1)$ follows with $L = \log_q N$.

Extend to large client space, $m \geq b$. Recall that the client space, denoted by m , is the maximal number of bits that the ORAM can store locally. For readability, we supposed that the ORAM locally stores $O(1)$ number of blocks, i.e., $m = \Theta(b)$. However, real-world systems more often incorporate a larger local storage. E.g., in a physical processor, the cache is often much larger than its cache line. Recall that for any $m \geq b$, the lower bound is $\Omega(\log(\frac{Nw}{m}) / (1 + \log(\frac{b}{w})))$ [KL21].

Our ORAM can be easily modified to match the lower bound, including m . Let $M = m/w$ be the number of words the client stores. Modify Construction 4.2 as below.

- Store the first $\ell_0 = \lfloor \log_q(2M) \rfloor$ levels locally the client space. Hence, the hash tables $T_0, T_1, \dots, T_{\ell_0-1}$ are always accessed locally.

The above modification works because the first ℓ_0 levels take at most $2q^{\ell_0} \leq M$ words in space. Hence, the modified ORAM accesses physical block only in $L - \ell_0$ levels, and the I/O overhead is $O(L - \ell_0) = O(\log_q N - \log_q(m/w)) = O(\log(Nw/m)/\log q)$. The overhead matches the lower bound up to a constant factor when $B = b/w = \Omega(\log^{1+\epsilon} N)$ is mildly large and $q = B/\log N$. $\square$

Remark C.4 (Remove the assumption of perfect powers). *In the above Construction 4.2, we assumed that the number of client words N is a perfect power of q . While we may choose q in the range $[\sqrt{B}, (B/\log N) + 1]$, rounding N up to the next perfect power incurs a large constant, which is undesirable. To address it, we show how to modify the construction for any N at a high level.*

Let q be a parameter to be chosen in the same range, $[\sqrt{B}, (B/\log N) + 1]$. Set $L = \lfloor \log_q N \rfloor$. The levels are the same, from $T_0, T_1, \dots$ to T_{L-1}, T_L , and for each $i < L$, the level T_i is the same (which stores $(q-1) \cdot q^i$ elements at most). The only difference is the largest level, T_L , which still stores at most N elements, but now we have $2^L \leq N < 2^{L+1}$. Hence, similar to the smaller levels, level L is associated with a $\text{Load}_L \in \{0, 1, \dots, \lceil N/2^L \rceil\}$. For every 2^L number of logical accesses, if T_L is not yet full (that is, Load_L is less than $\lceil N/2^L \rceil$), we rebuild 2^L elements into T_L and increment Load_L . Otherwise, T_L is full, we compact the $2^L + N$ elements (where 2^L from the smaller levels and N from T_L) down to N elements and build them into the new T_L .

The cost is the same for levels $i < L$. For level L , there are two differences. First, when T_L is not yet full ($\text{Load}_L < \lceil N/2^L \rceil$), extracting T_L is faster, and thus the cost is already bounded in the analysis. Second, the compaction works on $2^L + N$ elements at most, which is also bounded by $2N$ in the above analysis. Hence, Theorem 4.3 holds for the modified construction.

Corollary C.5 (Corollary 4.4, restated). *Let N be the total number of cells in the logical memory, and let $w \geq \log N$. Suppose that $B = b/w \geq \log^2 N$. Then, Construction 4.2 obviously implements Functionality 4.1 and takes I/O overhead $4 \cdot \lfloor \frac{\log N}{\log q} \rfloor + 2 + o(1)$, where $q = B/\log N + 1$, where the failure probability is $O(\frac{\log N}{\log q}) \cdot e^{-B/6} \cdot T$ over T logical accesses.*

Further suppose that $B = \omega(\log^4 N)$, then the I/O overhead is reduced to $2 \cdot \lfloor \frac{\log N}{\log q} \rfloor + 4 + o(1)$, where $q = B/\log^2 N$.

Proof. The proof comes directly from Eq. (3). To see why, with $q = B/\log N + 1$, Eq. (3) simplifies to $4L + 2 + 2/q + 20/(q-1) + 16/B = 4L + 2 + o(1)$, where $L = \lfloor \log N / \log q \rfloor$, where the rounding down follows by Remark C.4.

Further, with $B = \omega(\log^4 N)$, we have $q = B/\log^2 N = \omega(\log^2 N)$. By $(q-1)/B \leq 1/\log^2 N$, in Equation (3), we have $(L(q-1) \cdot \log N)/B \leq L \log N / \log^2 N = o(1)$, which yields $2L + 4 + o(1)$ as claimed. $\square$